

# ビルの飾り付け4 (Building 4) 解説

細川 寛晃

# 問題概要

- 長さ  $2N$  の整数列  $A, B$  がある
- 各  $i$  に対し  $A_i, B_i$  のどちらかを選んで、以下の条件を満たすようにできるか？
  - $A$  から  $N$  個の項を選んでいる
  - 選んだ項で作った、長さ  $2N$  の数列が(広義)単調増加
- できる場合はそのような選び方を構成せよ

# 動的計画法(DP)

- 「○○な条件を満たすような選び方の数を求めよ」のような問題がDPで解けることが往々にしてある
- 今回は存在するかどうか判定するだけだが、DPによるアプローチを考えてみる

# 必要な状態は？

- 前から順に、A と B のどちらの項を選ぶかを決めていく
  - 「A から N 個の項を選ぶ」という条件がある
- 今までに選んだ A の個数を状態として持っておく必要がありそう
- 「単調増加」の条件
    - 今選ぼうとしている項が、一つ前の項よりも大きければOK
    - 「一つ前の項を A と B のどちらから選んだか」によって、今選べる項が変わってくる
- 「一つ前の項を A と B のどちらから選んだか」も状態として必要

# 小課題1 (11点)

- bool値のDPをします
- $dp[i][j][A \text{ (or } B)] :=$  ビル  $i$  まで見て  $A$  から  $j$  個選んでおり、最後に選んだのが  $A$  (or  $B$ ) である状態にできるかどうか
- 遷移の例 ( $A$  から  $A$  のとき):  
$$A_i \leq A_{i+1} \text{ であれば } dp[i][j][A] \rightarrow dp[i+1][j+1][A]$$
のように行う
  - $A$  から  $B$ 、 $B$  から  $A$ 、 $B$  から  $B$  の遷移も同様
- $dp[2N][N][A]$  または  $dp[2N][N][B]$  が true ならば条件を満たす選び方が存在
- 計算量は  $O(N^2)$



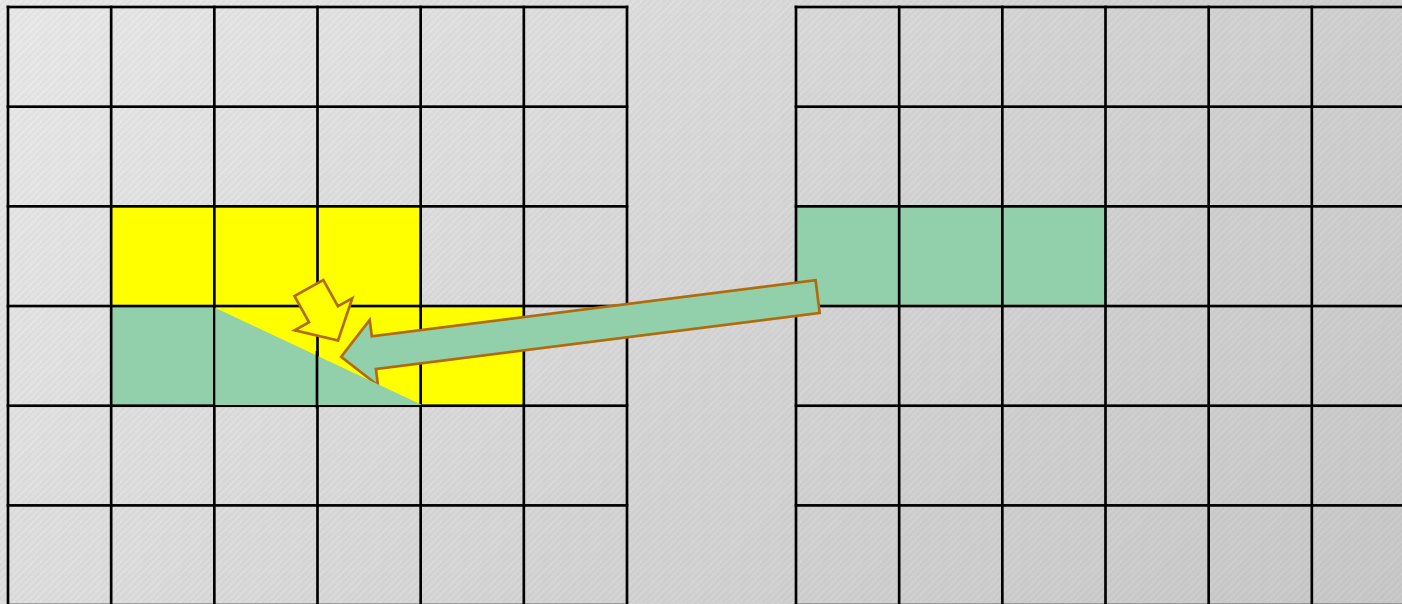
# 解の復元パート

- 後ろ( $dp[2N][N][A \text{ or } B]$ )から見ていく
- 値が true である状態からは1つ戻っても true である状態が存在
  - 例えば  $dp[i][j][A]$  が true なら  $dp[i-1][j-1][A]$  と  $dp[i-1][j-1][B]$  の少なくともいずれかは true
  - その戻り方ができるかは要確認
    - 例えば A から B に戻るときは  $B_{i-1} \leq A_i$  でなければならない
- true である状態をたどっていけば、それは条件を満たす選び方になっている

# 遷移を観察

- 実は  $dp[i][j][A \text{ or } B]$  が true となる  $j$  は区間になっている！
  - 証明は後日公開します(帰納的/非帰納的な証明が存在します)
- この区間を計算するDPをすれば解けそう

# 概念図





# 小課題2 (89点)

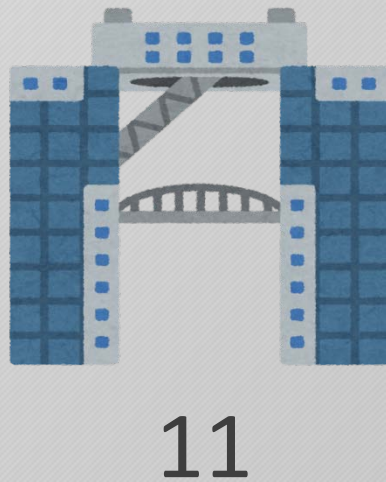
$dp'[i][A \text{ or } B] := dp[i][j][A \text{ or } B]$  が true となるような  $j$  の区間

- としてDPを行う
- 遷移の例:
- $A_i \leq A_{i+1}$  であれば  $dp'[i+1][A]$  の区間に  $dp'[i][A]$  の区間をマージ
  - $A$  を選ぶときは区間が1つずれるので注意
- 計算量は  $O(N)$

# 解の復元パート

- 先ほどと同様、後ろ( $dp'[2N][A \text{ or } B]$ )から見ていく
- Aから選んだ個数( $j$ とする)を管理しながら、 $dp'[i][A \text{ or } B]$ の区間内に $j$ が含まれているような状態をたどっていけばOK

# 得点分布







美味しい美味しいハンバーグ



# 概要

- 長方形(ハンバーグ)がたくさんあるので、K個の点で覆えるか？
- $N \leq 200,000$
- $K \leq 4$



$$K = 1$$

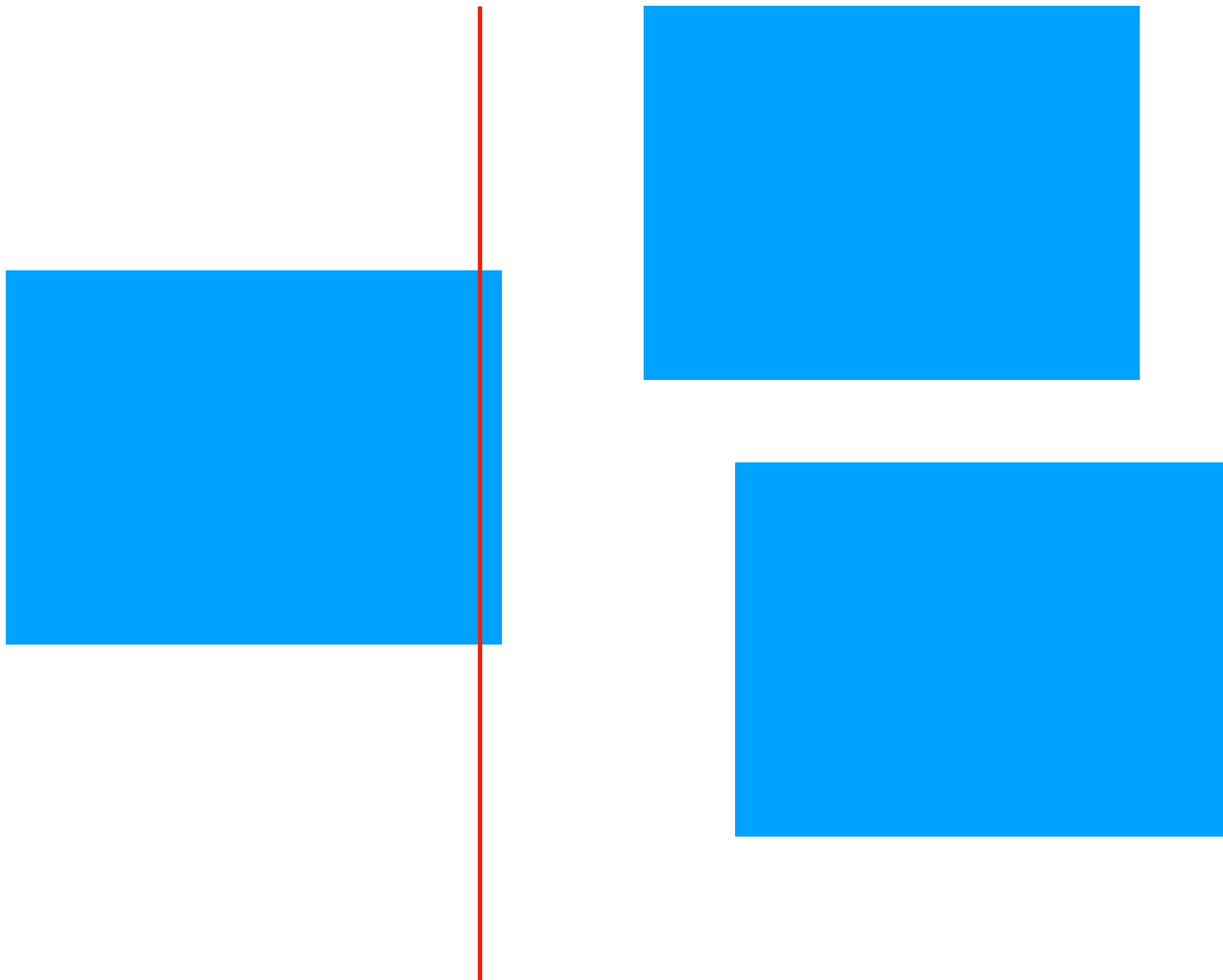
- ・ 絶対に  $\max L_i \leq \min R_i$  かつ  $\max D_i \leq \min U_i$  が満たされていて、答えは  $(\max L_i, \max D_i)$  などです

$$K = 2,3$$

- 重要: 長方形の右辺のうち最も左のもの

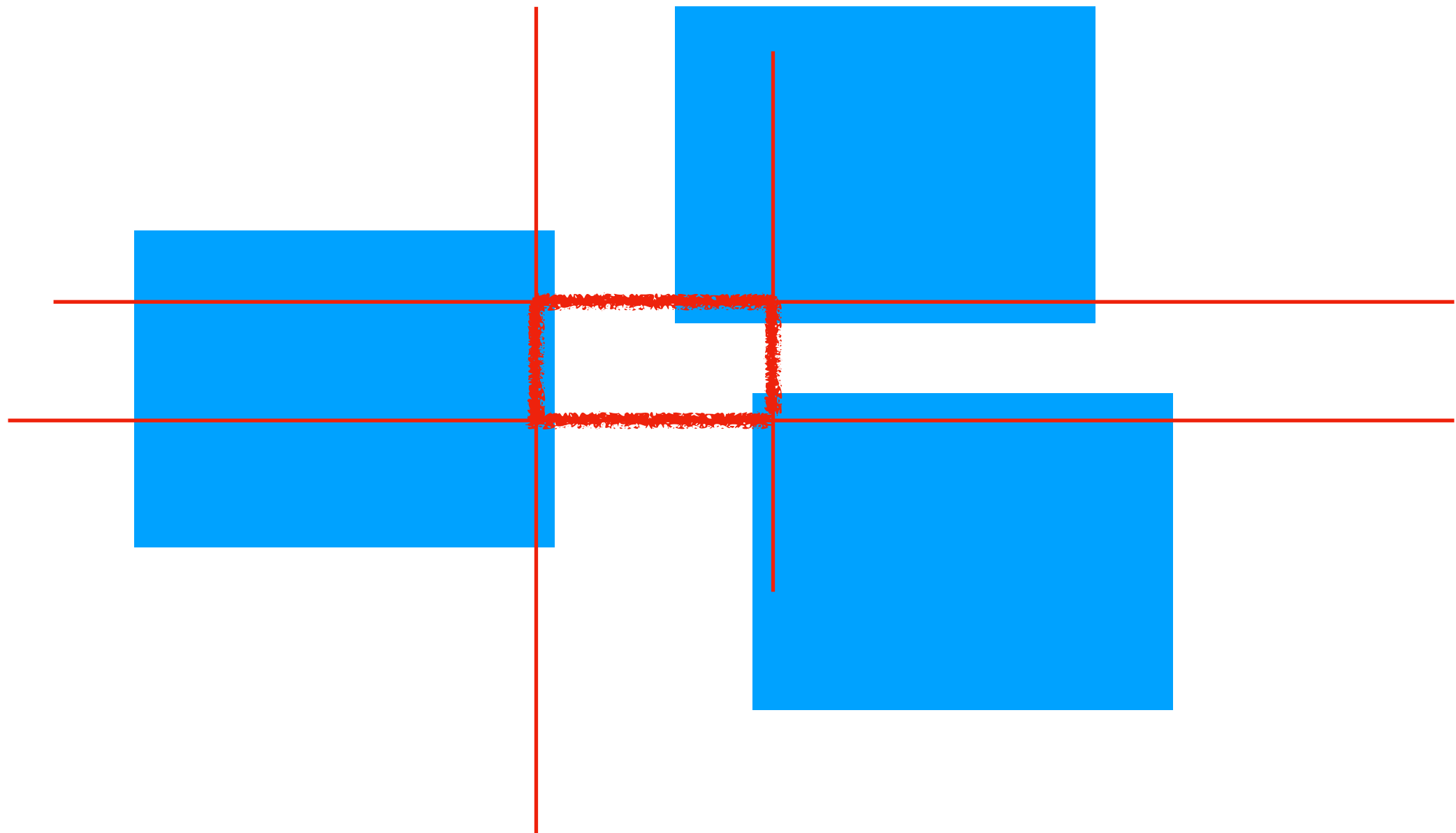
$$K = 2,3$$

- 重要: 長方形の右辺のうち最も左のもの



$$K = 2,3$$

- 上下左右についてやると本質の長方形が生まれる





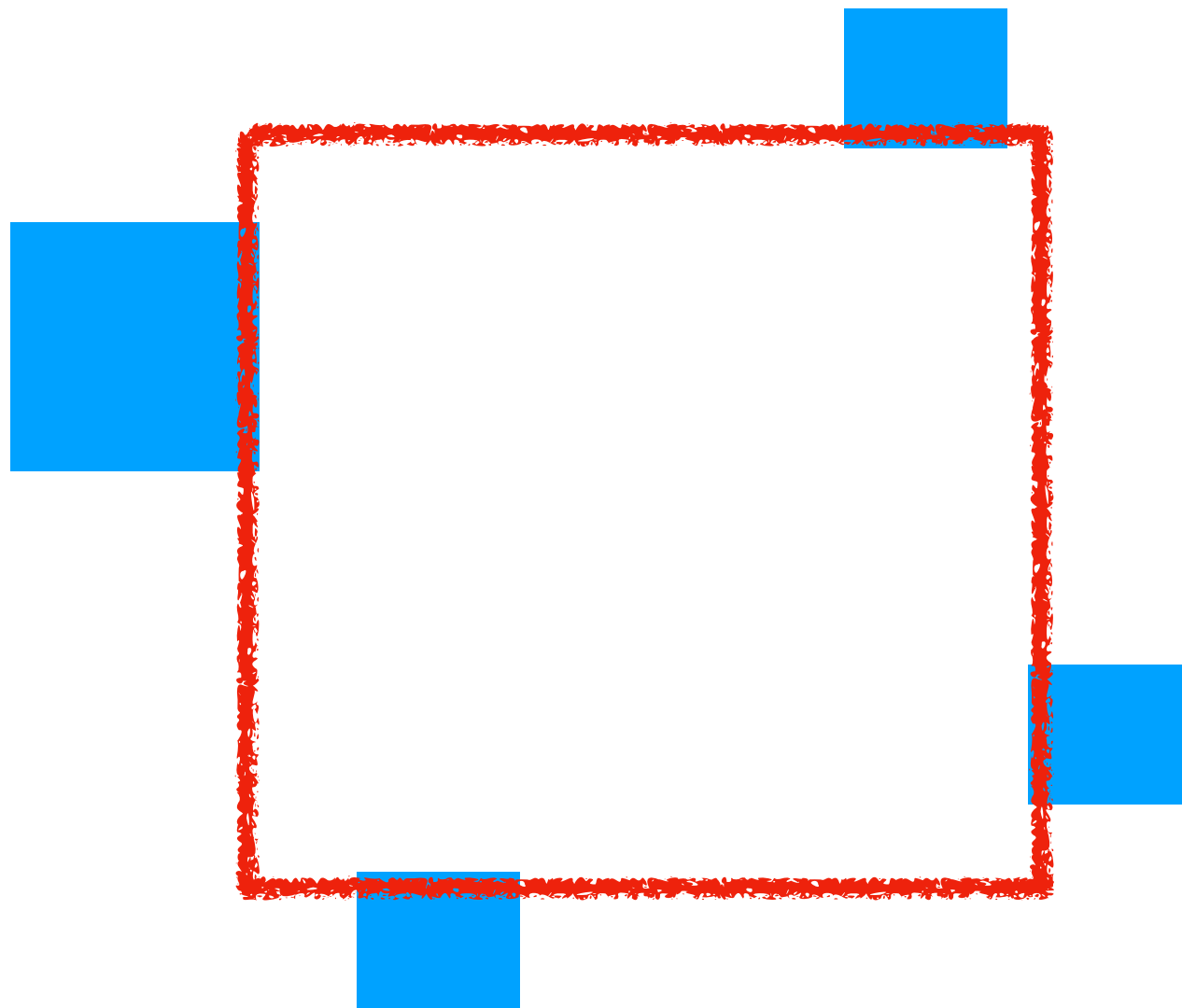
$$K = 2,3$$

- 上下左右についてやると本質の長方形が生まれる
- どの辺にも1個は点に乗っている必要がある
  - $K \leq 3$ なら四隅のどれかは使う
  - 全探索でAC！



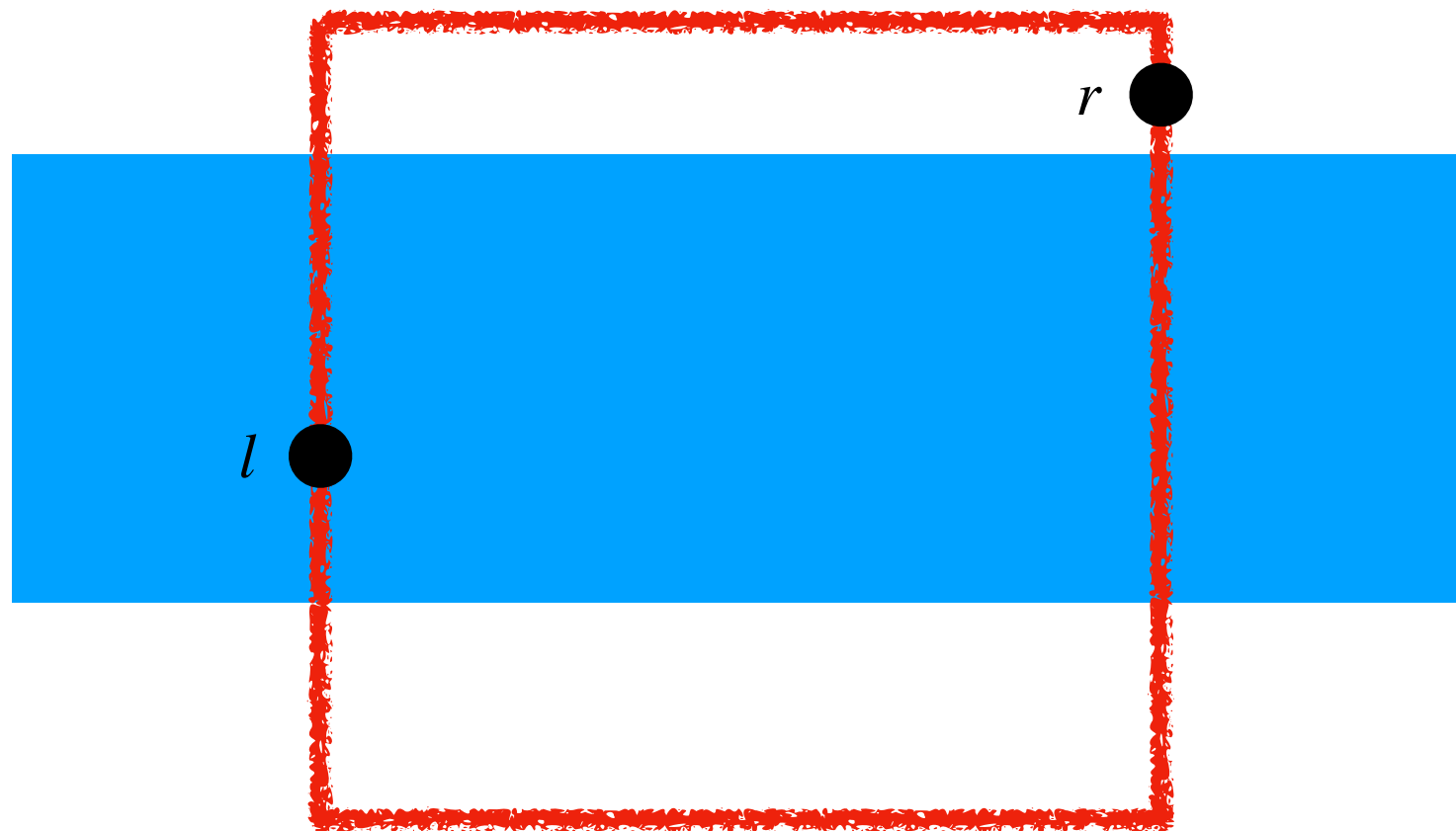
$$K = 4$$

- 4点とも辺の上のケースがある！



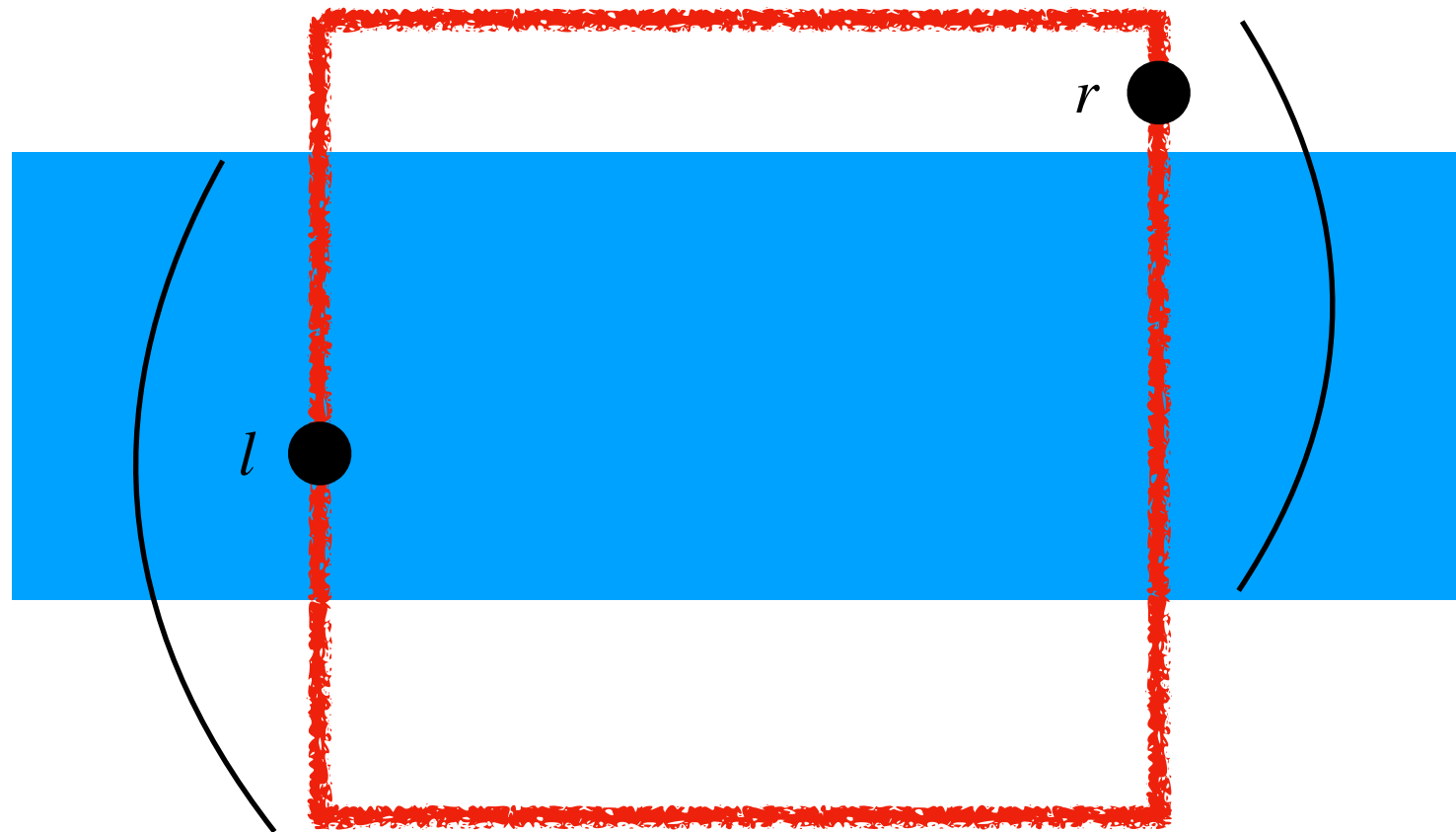
# 本質(考察)

- やばいのはぶちぬく長方形
- $(D_1 \leq l \leq U_1)$  or  $(D_2 \leq r \leq U_2)$  の形の制約になり、むずそう

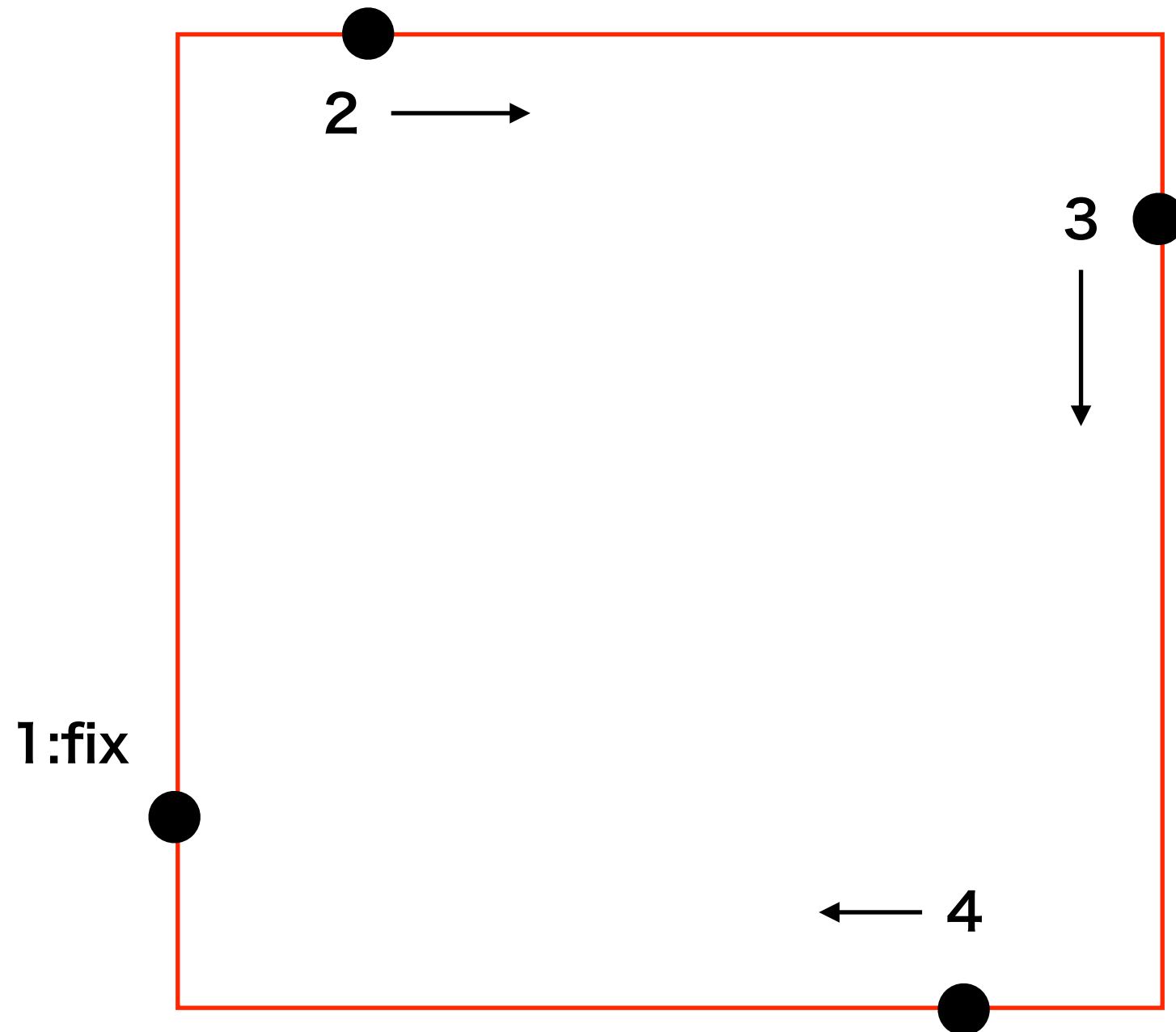


# 本質(考察)

- ・ 実は全部の長方形に対して  $l \leq U_1, D_2 \leq r$  を仮定できる

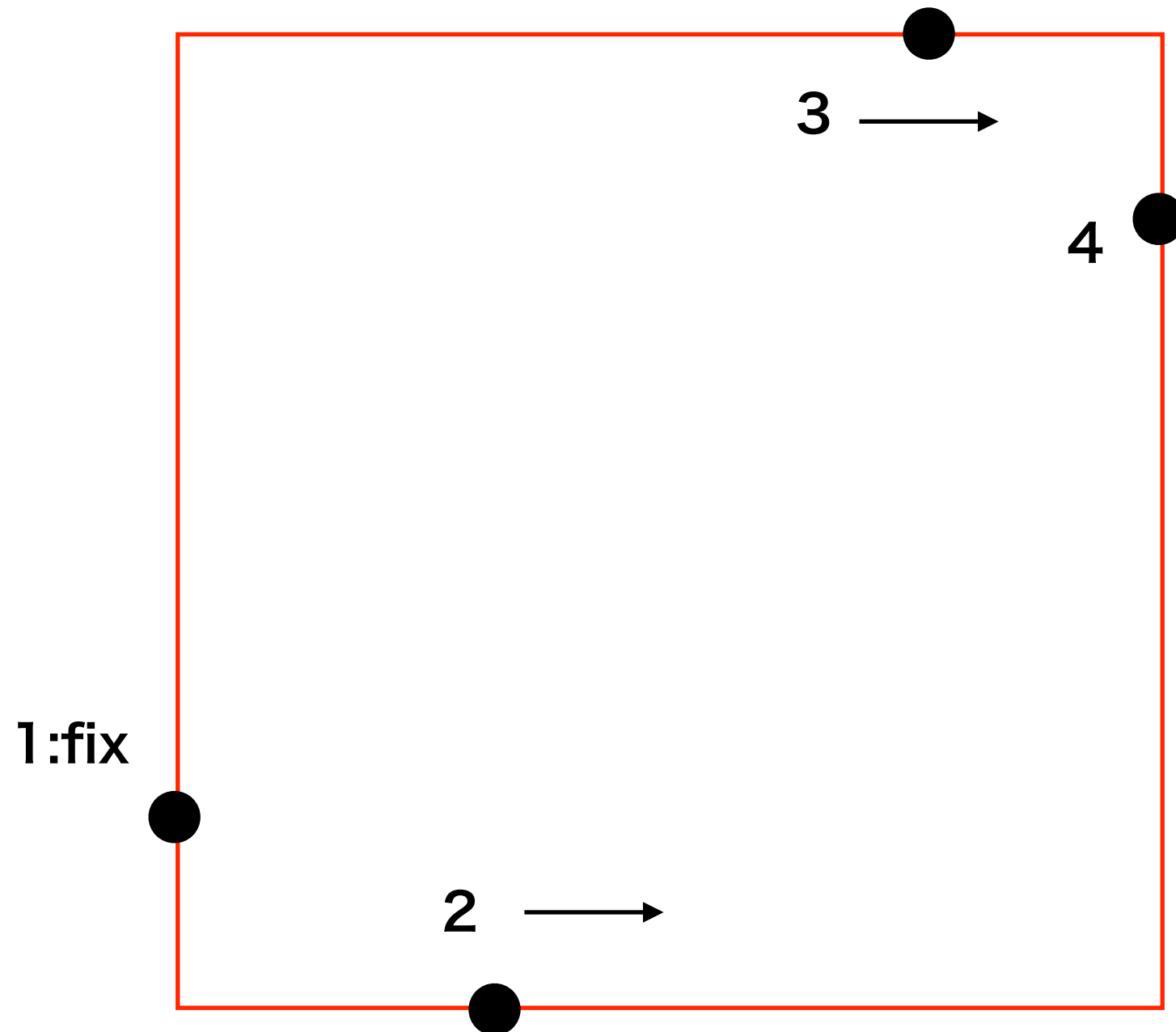


# 本質パターン1





# 本質パターン2



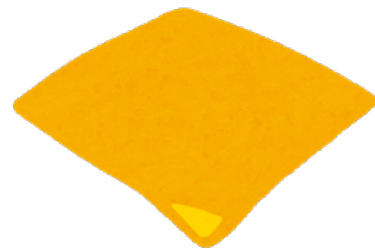
# 本質(実装)

- どうやって実装すればいいか調べてみました！
- チューター陣は綺麗に実装する方法がわかりませんでした！
- チューター解(yosupo): 355行 9759bytes (テンプレ14行)
- いかがでしたか？

# 得点分布



2



3



6



21

掃除

SWEEPING

# 概要

以下の4種類のクエリが飛んでくる(もとかからあるホコリは追加クエリによるものと同一視した)

- $i$ 番目のホコリの座標を求める
- $x < N - L$ かつ $y \leq L$ を満たす $(x, y)$ にあるホコリを $(N - L, y)$ に移動する
- $x \leq L$ かつ $y < N - L$ を満たす $(x, y)$ にあるホコリを $(x, N - L)$ に移動する
- $(x, y)$ にホコリを追加する

# 小課題1

- $M \leq 2000$
- $Q \leq 5000$
- 愚直にシミュレーションをすれば解ける
- $O((M + Q)Q)$
- 1点



# 小課題2

- $T_j = 1, 2, 4$
- y座標が変化しない
- あるホコリのx座標は、そのホコリが発生してから移動クエリで  $y \leq L$  を満たすもののうち、 $L$  が最小のものによって決定される

# 小課題2

- あらかじめy座標の値によってクエリをソートして、y座標が小さいものから順番に処理していく
- 移動クエリの管理はRMQとして扱えるので、Segment Treeで行う
- $O(M + Q \log Q)$
- 10点

# 小課題3

- $T_j = 1, 2, 3$
- $X_j \leq X_{j+1}$
- $Y_j \geq Y_{j+1}$
- このとき、クエリの性質を観察すると、隣り合うホコリは(同じx座標・y座標になることはあっても)、x座標・y座標が反転しない

# 小課題3

- x座標・y座標が反転しないので、ホコリを順序付けて管理できる
- 移動クエリの効果があるのはこの順序上の連続した区間
- どこからどこまで効果があるのかは二分探索で求められる
- 更新は遅延評価Segment Tree

# 小課題3

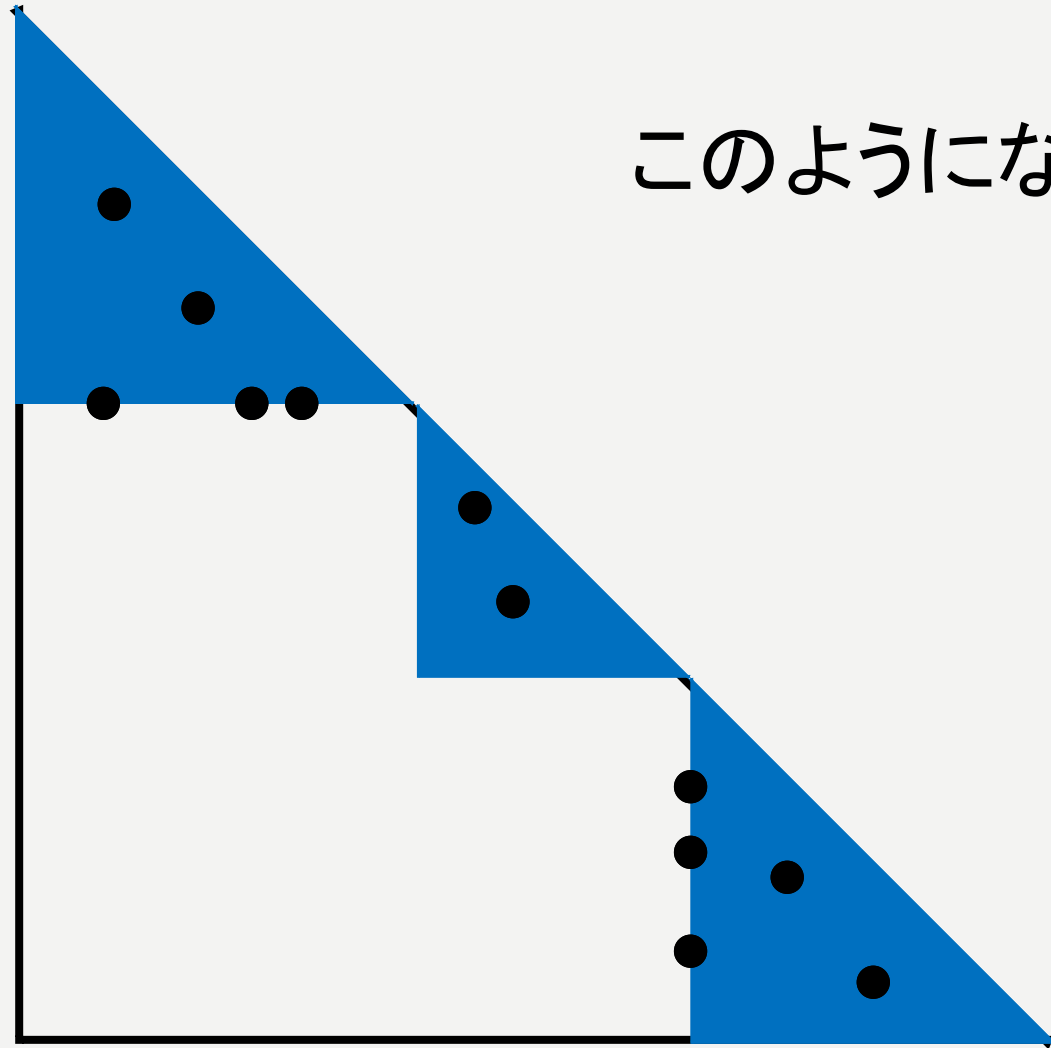
- 各クエリを  $O(\log^2 M)$  で処理できるので、全体で  $O(Q \log^2 M)$  で解ける
- 11点

# 小課題4

- $T_j = 1, 2, 3$
- 追加クエリがない
- 追加クエリがないとき、ホコリが存在しうる場所は、斜辺が部屋の斜めの壁の一部である互いに重ならない直角二等辺三角形の集合となる

# 小課題4

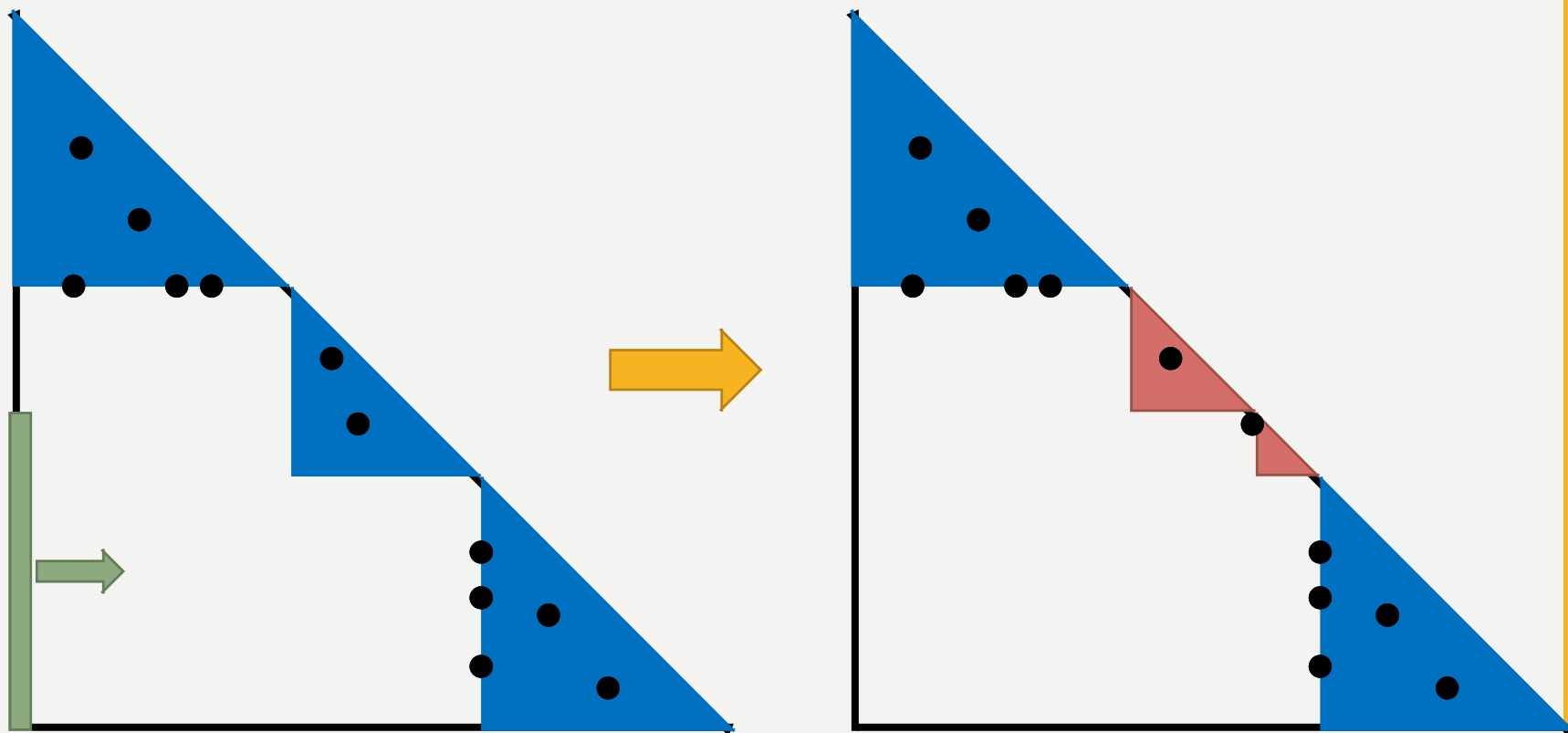
このようになる





# 小課題4

- また、掃除のクエリ1回につき、1つの三角形領域は2つに分割される



# 小課題4

- 全体として、1つの三角形領域が $O(Q)$ 個の三角形領域に分割されることとなる
- ある三角形領域を $X$ 個の木コリを持つ三角形領域と $Y$ 個の木コリを持つ三角形領域に分割するとき、 $O(\min(X, Y) \log(X + Y))$ 程度で分割できる
- マージテクの逆を行うことで、全体で $O(Q + M \log M \log Q)$

# 小課題4

- 連結リストのようなイメージでうまくホコリを管理してやると全体で $O(Q + M \log Q)$ で計算できる
- 小課題5のためにはこっちを使う
- どっちにしても64点

# 小課題5①

- 小課題4の解法に対してクエリ分割統治を行う
- 各移動クエリを $O(Q)$ 回行うことになる
- 全体の計算量は
$$O((M + Q) \log(M + Q) \log Q)$$
- 100点!

# 小課題5②

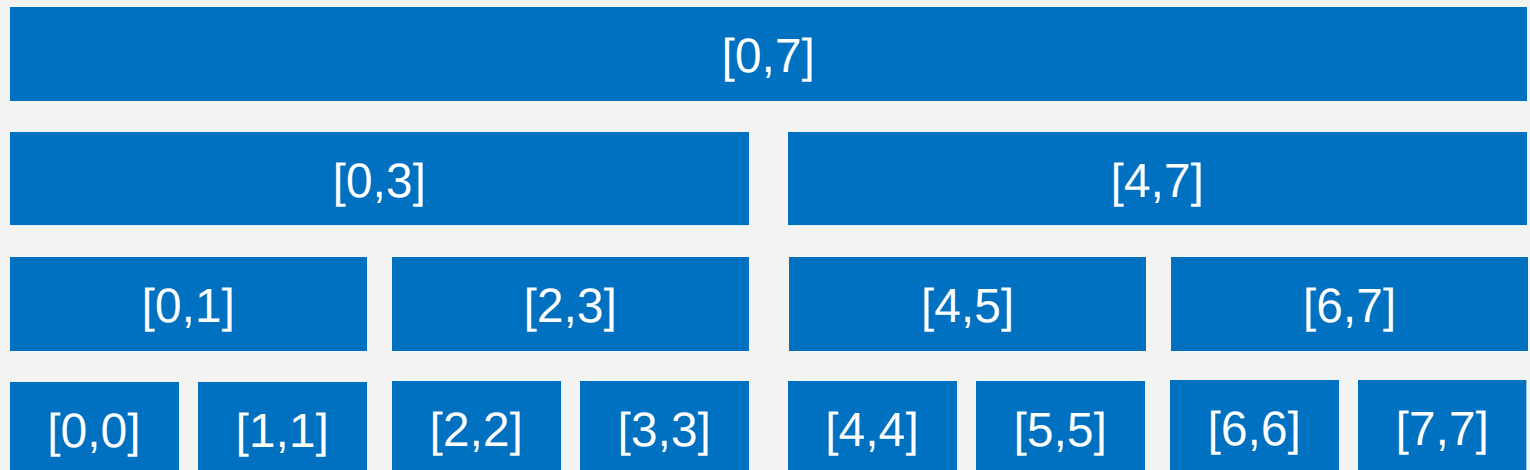
まず、2次元座標から数直線上の区間に問題を変換する

- 座標 $(x, y) \Rightarrow$ 区間 $[y, N - x]$
- $x < N - L$ かつ $y \leq L$ のホコリを $(N - L, y)$ に移動する  
 $\Rightarrow a \leq L \leq b$ となる区間 $[a, b]$ を $[a, L]$ に変更する
- $x \leq L$ かつ $y < N - L$ のホコリを $(x, N - L)$ に移動する  
 $\Rightarrow a \leq N - L \leq b$ となる区間 $[a, b]$ を $[N - L, b]$ に変更する

これを1次元の区間木に乗せてクエリを処理する

# 小課題5②

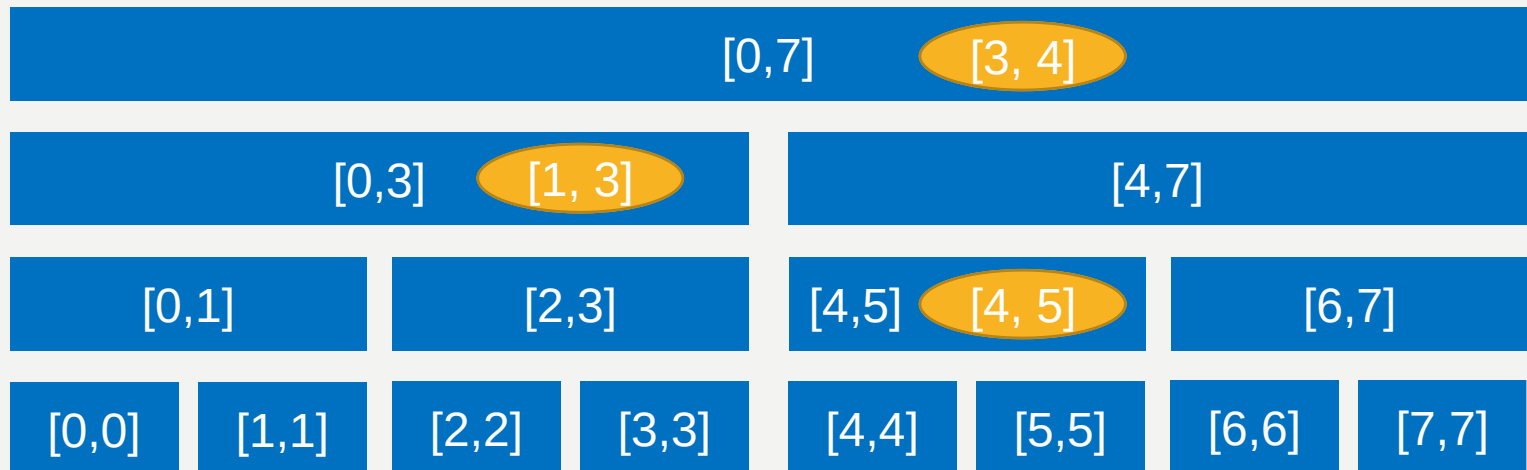
- 1次元の区間木とは  
区間を管理するSegment Tree



# 小課題5②

- 1次元の区間木とは

各区間はその区間を丸々収めることができるノードのうち最も小さいものに格納される



## 小課題5②

- 移動のクエリのたびに、各ホコリは同じノードにとどまるか、子のノードへと降りていく
- つまり、各ノードでかかる計算量をホコリ当たり  $O(M + Q)$  程度にしてやればよい
- 各ノード内でUnion Find等をもってホコリを管理してやれば、これは達成できる



# 小課題5②

- 区間木の深さは $O(\log(M + Q))$   
(座標圧縮しないと $O(\log N)$ )
- それぞれのノード内での操作ある深さのノードの合計で $O((M + Q)\log(M + Q))$
- よって全体の計算量は $O((M + Q)\log^2(M + Q))$
- 100点!

# カメレオンの恋

坂部圭哉



# 注意



- ▶ 小課題1が解けるコードと  
小課題2が解けるコードがあるとき  
場合分けして1つにまとめる必要は  
無い。
- ▶ 2回提出すればOK

# 準備



- ▶ カメレオンを頂点としたグラフを考える
- ▶ それらの間に2種類の辺がある
  - ▶ 「同色」という無向辺
  - ▶ 「好み」という有向辺
- ▶ これは**二部グラフ**
- ▶ 無向辺で結ばれたペアを答える問題

# 小課題1



- ▶ 「両思い」 のとき、その辺は無視
- ▶ 無向辺だけのグラフとみなせる
- ▶ 二分探索で解ける

## 小課題2



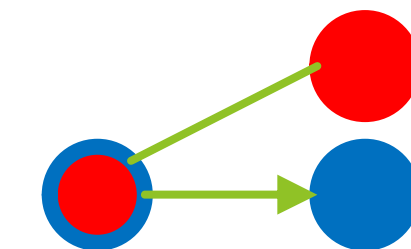
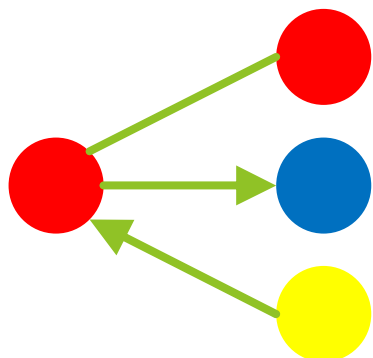
- ▶ 2匹でQuery
  - ▶ 戻り値が1 のとき、その間に辺あり
  - ▶ 戻り値が2 のとき、その間に辺なし
- ▶ 辺の種類を同定する方法が分からない...
  - ▶ とりあえず辺の種類を全探索 → 解ける

## 小課題3

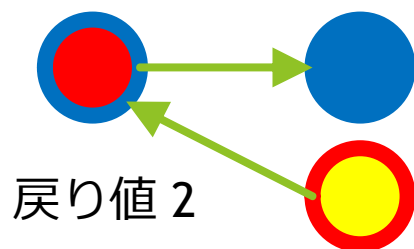


- ▶  $N^2$ のペアで投げて、辺を全列挙
- ▶ 辺の種類を同定する方法  $O(N)$ 
  - ▶ 各頂点は、次数1(両思い)または3
  - ▶ 次数1 → その辺は無向辺
  - ▶ 次数3 → 3つ組で投げて有向辺を列挙  
(次ページに図)

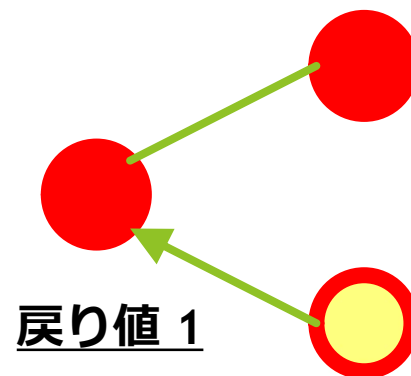
# 小課題3



戻り値 2



戻り値 2



戻り値 1



## 小課題4



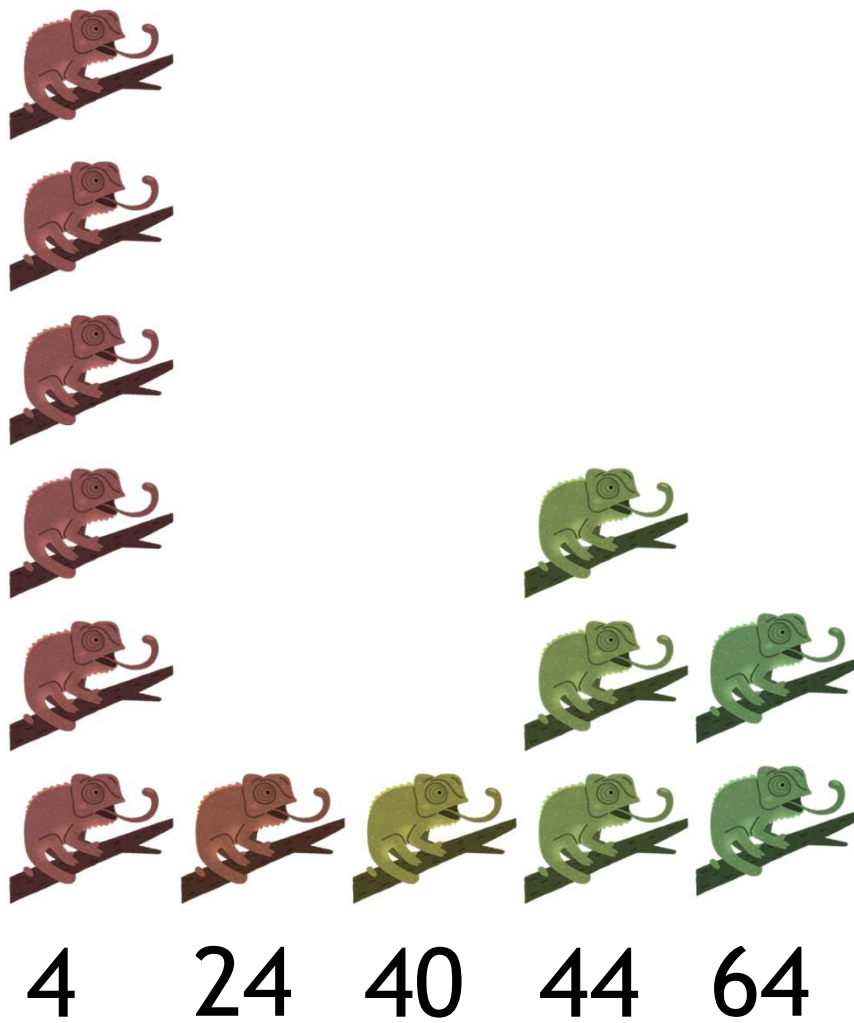
- ▶ 性別が分かっているので  
二分探索で辺を列挙  
(戻り値 = 頂点数  $\rightarrow$  辺がない)  
(戻り値 < 頂点数  $\rightarrow$  辺がある)
- ▶ 辺の種類の同定は小課題3と同じ

## 小課題5(満点)



- ▶ 1頂点ずつ見て  
すでに見た頂点との間の辺を列挙
- ▶ グループ内に辺が無いように  
すでに見た頂点を2グループに分割  
(二部グラフなので可能)
- ▶ 性別が分からなくても  
二分探索で辺が分かる

# 得点分布



# Making Friends on Joitter is Fun

解説

tozangezan

Making Friends on Joitter is Fun?



# Making Friends is Fun (2014)



Japanese Olympiad in Informatics 2013/2014  
Spring Training Camp/Qualifying Trial  
March 19–25, 2014, Komaba/Yoyogi, Tokyo

Contest Day 2 – Making Friends is Fun

## 友だちをつくろう (Making Friends is Fun)

あなたは歴史の裏舞台で活躍するエージェントであり、世界の平和に向けて日々活動をしている。この世界には  $N$  個の国があり、それぞれ 1 から  $N$  までの異なる番号がふられている。これらの  $N$  個の国々の間にできる限り友好な関係を築いてもらうことがあなたの目的である。あなたはエージェントの仕事の計画を立てるため、現在の国際関係を表す図を描いた。

あなたは大きな画用紙を一枚用意し、まずそこにそれぞれの国を表す  $N$  個の点を打った。次に、現在の国際関係を表すために、2 つの国を結ぶ矢印を  $M$  本描いた。国  $a$  を表す点から別の国  $b$  を表す点へと向かう矢印は、「現在、国  $a$  が国  $b$  に大使を派遣している」ということを表す。以下では、国  $a$  を表す点から国  $b$  を表す点へと向かう矢印を矢印  $(a, b)$  と呼ぶ。こうして描いた  $N$  個の点と  $M$  本の矢印が現在の国際関係を表す図である。

国同士の友好関係のきっかけとして、2 国間での友好条約締結会議（以下では単に「会議」という）を行うことを考えよう。ある 2 つの国  $p, q$  が会議を行うためには、両方の国に大使を派遣しているような国  $x$  が仲介として必要である。そして、会議を行った後にそれぞれの国は相手国に大使を派遣する。すなわち、国  $p$  と国  $q$  が会議を行うためには、矢印  $(p, x)$  と矢印  $(q, x)$  があるような国  $x$  が存在していなければならない。

# Joitter (2011)



Japanese Olympiad in Informatics 2010/2011

Qualifying Trial

April 23–24, May 3–5, 2011, Tokyo/Osaka

Contest Day 1 – Joitter

---

## ジョイッター (Joitter)

ジョイッター (Joitter) は、短い日記の気軽な投稿や写真の共有を通して、知り合いとのインターネット上でのコミュニケーションをより快適にする、話題沸騰中のソーシャル・ネットワークキング・サービス (SNS) である。

ジョイッターでは、自分以外のユーザを「友人」というリストに登録することができる。あるユーザ A があるユーザ B を「友人」として登録しようとする、ユーザ B に通知が届く。ユーザ B がこれに許可することで、2 人は互いに「友人」として登録される。これを 1 回の「友人」登録と考える。「友人」登録には、なぜか 2 人のユーザに依存したコストがかかる。ユーザ A とユーザ B が互いに「友人」でありユーザ B とユーザ C が互いに「友人」であっても、ユーザ A とユーザ C が互いに「友人」となるとは限らない。

ジョイッターでは、ユーザは日記の公開設定を以下の 3 種類のいずれかに設定できる。

- (1) 「友人」にのみ公開する。
- (2) 「友人」あるいは「友人」の「友人」であるユーザにのみ公開する。



# Making Friends is Fun (2014)



Japanese Olympiad in Informatics 2013/2014

Spring Training Camp/Qualifying Trial

March 19–25, 2014, Komaba, Tokyo, Tokyo

Contest Day 2 – Making Friends is Fun

## 友だちをつづろ (Making Friends is Fun)

あなたは歴史の舞台で活躍する記の気盛な投稿者であり、世界の平和に向かい奮い活動が続けようとする世界には  $N$  個の国があり、それぞれに異なる番号がふられていて、それぞれ  $M$  個 (SNS) の間、できる限り友好な関係を築いてもらうことがあなたの目的である。あなたはエージェントの仕事の計画を並べるため、現在の国際関係を表す図を描きたいというリストに登録することができる。あるユーザ A があなたは矢  $B$  を画用紙をと枚用意し、まずそれぞれ  $M$  国を通知  $M$  個の点を打ち、次に現在の国際関係を表すために、2 友の国を結ぶ矢印を  $M$  本描き、これを 国  $a$  を表す点から登録の国  $b$  を表す点へ登録は矢印が現在の国  $a$  国  $b$  国  $a$  大使を派遣している。そして、ユーザ A とを表す  $B$  国  $a$  を表す点から国  $B$  を表す点へ向かう矢印を矢印  $(a, b)$  と呼ぶ。ユーザ A と描いた  $N$  個の点と  $M$  本の矢印が現在の国際関係を表す図である。ユーザは日記の公開設定を以下の 3 種類のいずれかに設定できる。

国同士の友好関係のきっかけとして、2 国間での友好条約締結会議（以下では単に「会議」という）を行う。このことを考える。ある 2 国  $p, q$  が会議を行うためには、両方の国に大使を派遣しているような国  $x$  が仲介として必要である。そして、会議を行った後にそれぞれの国は相手国に大使を派遣する。すなわち、(2) 国  $p$  と国  $q$  が会議を行うためには、矢印  $(p, x)$  と矢印  $(q, x)$  があるような国  $x$  が存在していなければならない。



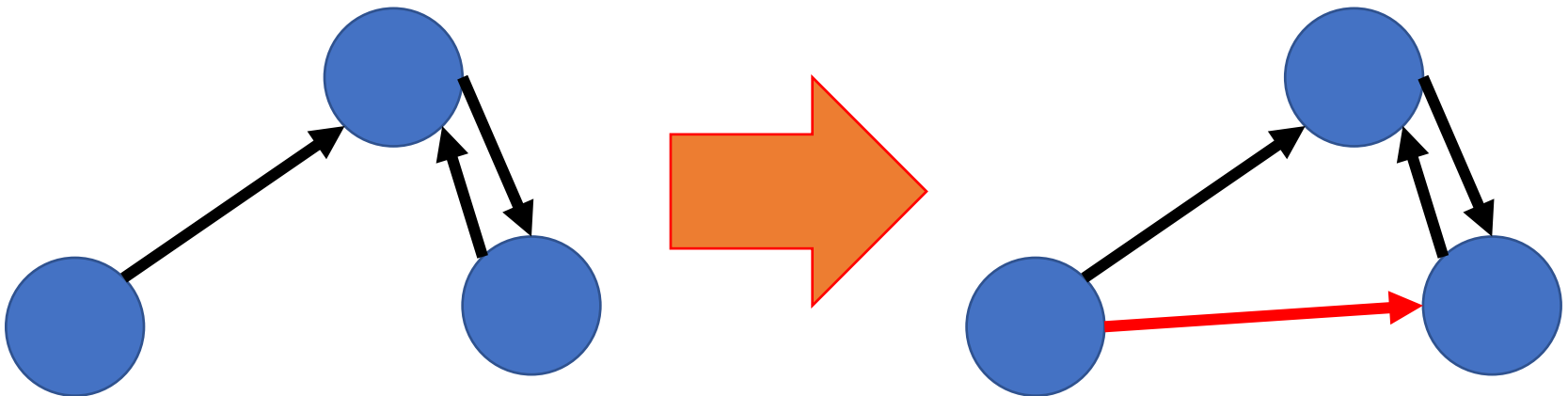


あなたは歴史を振り返る日記の気持で投稿で、世界の平和に向けて、世界にはN個の国があり、それぞれは異なる番号が振られており、M個(SNS)の間で、関係を築いてもらうことがあなたの目的である。ジェントの仕事の計画を並べる。国際関係を表す図を描きたいというリストに登録する。あるユーザAがある国を1枚用意し、それをそれぞれが国を通知する。また、次に現在の国際関係を表す図を描く。国を結ぶ矢印をM本の描いた国を表す点へ登録は矢印が現在の国Aが国Aに大使を派遣している。セザンAとをセザンB以下では、「国Aを表す点から国Bを表す点へ向かう矢印を矢印(a,b)と呼ぶ。ユーザAと描いたN個の点とM本の矢印が現在の国際関係を表す図である。では、ユーザは日記の公開設定を以下の3種類のいずれかに設定できる。

(2) 「友人」あるいは「友人」であるユーザにのみ公開する。

# 概要

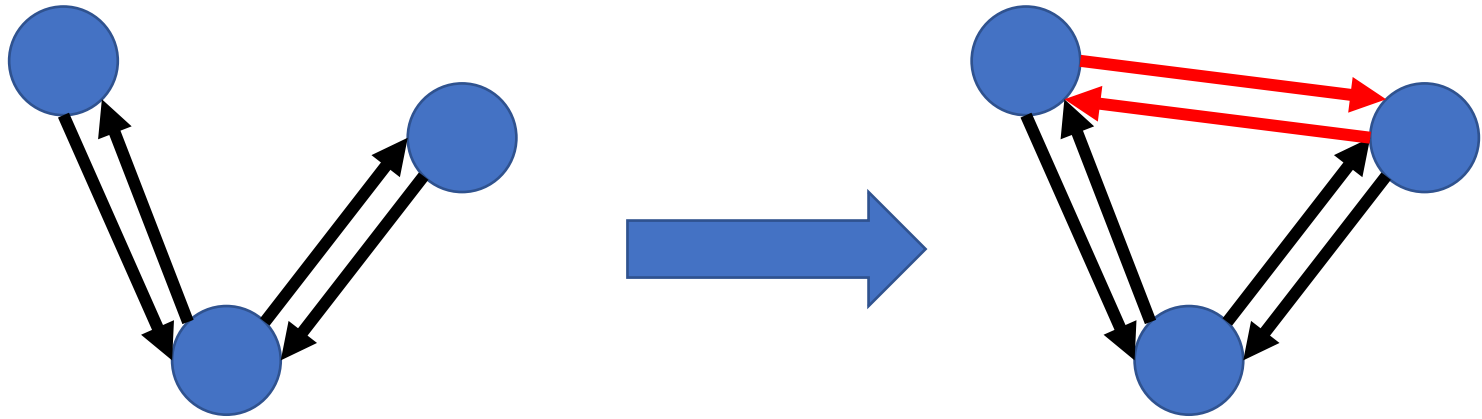
- 毎日有向グラフに1本ずつ辺が増えていく
- それぞれの日において以下の操作を繰り返したとき、最終的にグラフの辺の本数は何本になるか？



# 大前提

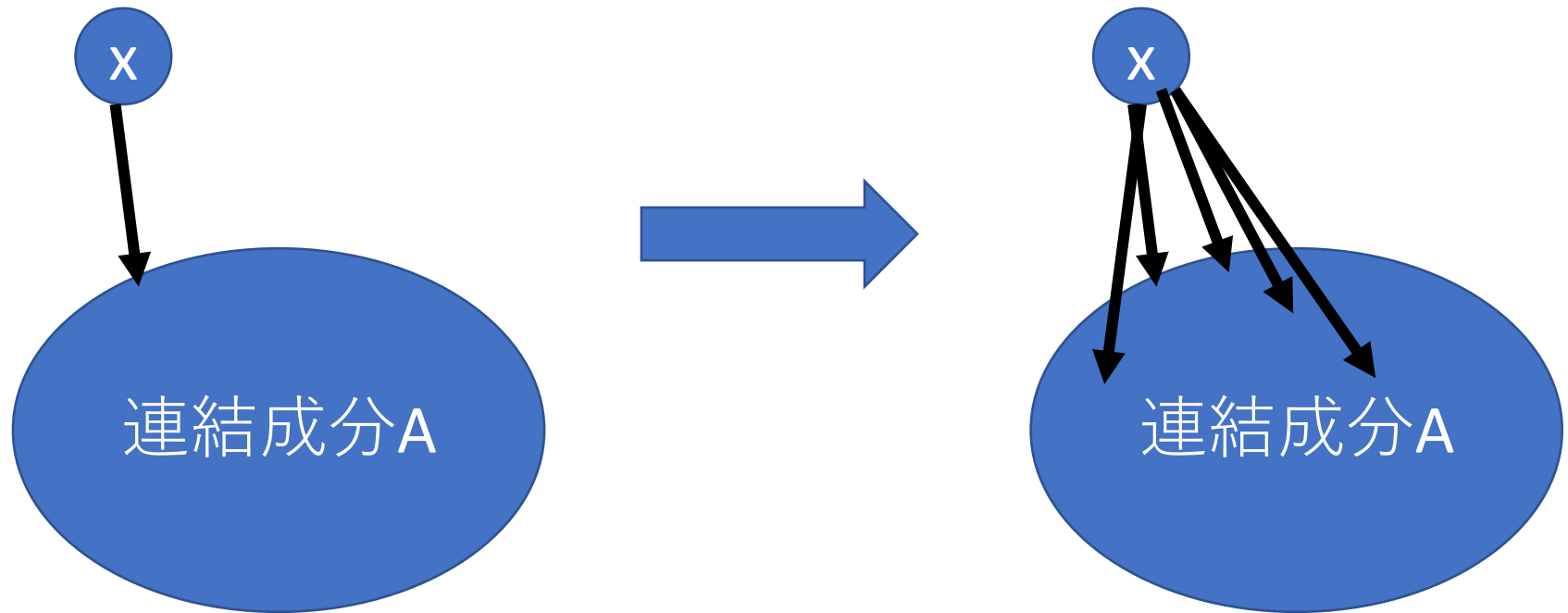
- 毎日に最初からグラフを作る必要はない
- 前日の答えを求めるのに使ったグラフに新たに増える辺を足していけば良い
- 問題文の通りに見つけては辺を増やす  
→  $O(N^3)$  1点

# 考察要素



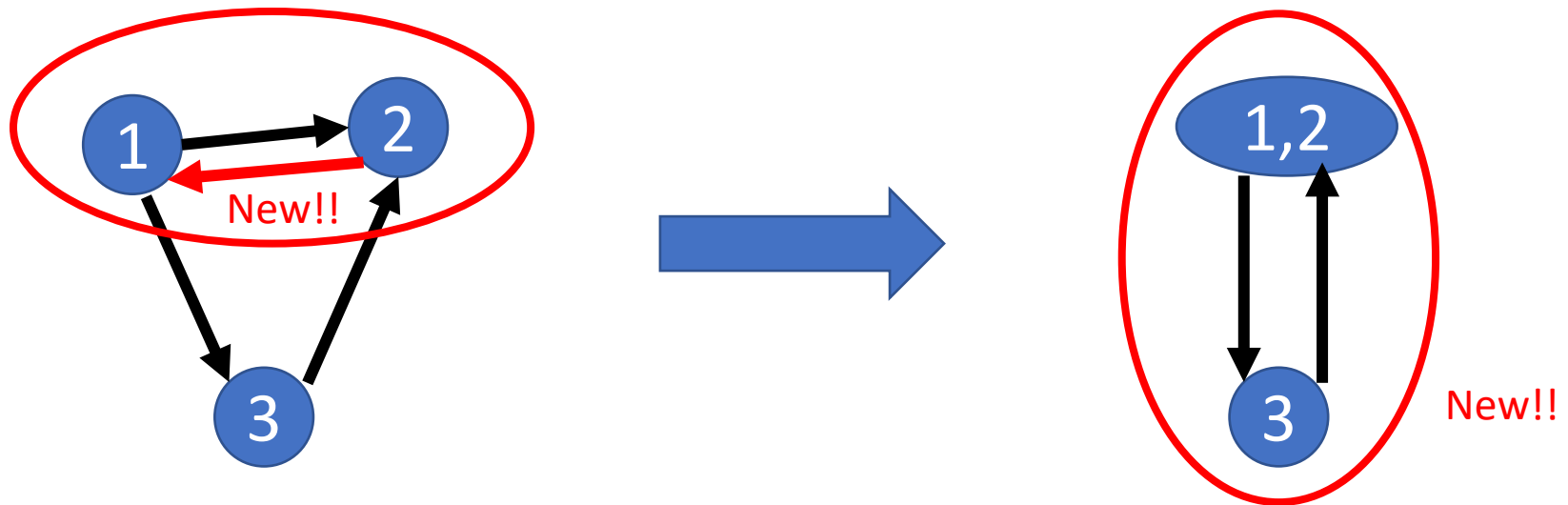
$A \leftrightarrow B \leftrightarrow C$  のように辺があると  $A \leftrightarrow C$  が結べる  
→ 両方向に辺がある二点は同一視して扱える！  
Union-find していくことにしよう

# 考察要素



ある頂点 $x$ から $y$ に辺があるとき、 $x$ から $y$ と同じ連結成分  
の他の頂点にも辺が生える  
 $(x, A)$ の組を辺の情報として管理すればよい

# 考察要素



1回マージすると連鎖的に何度もマージできることも...  
→ 繋げる頂点の組をBFSでマージしていく  
(DFSだと辺の情報が壊れるはず)

## 小課題2

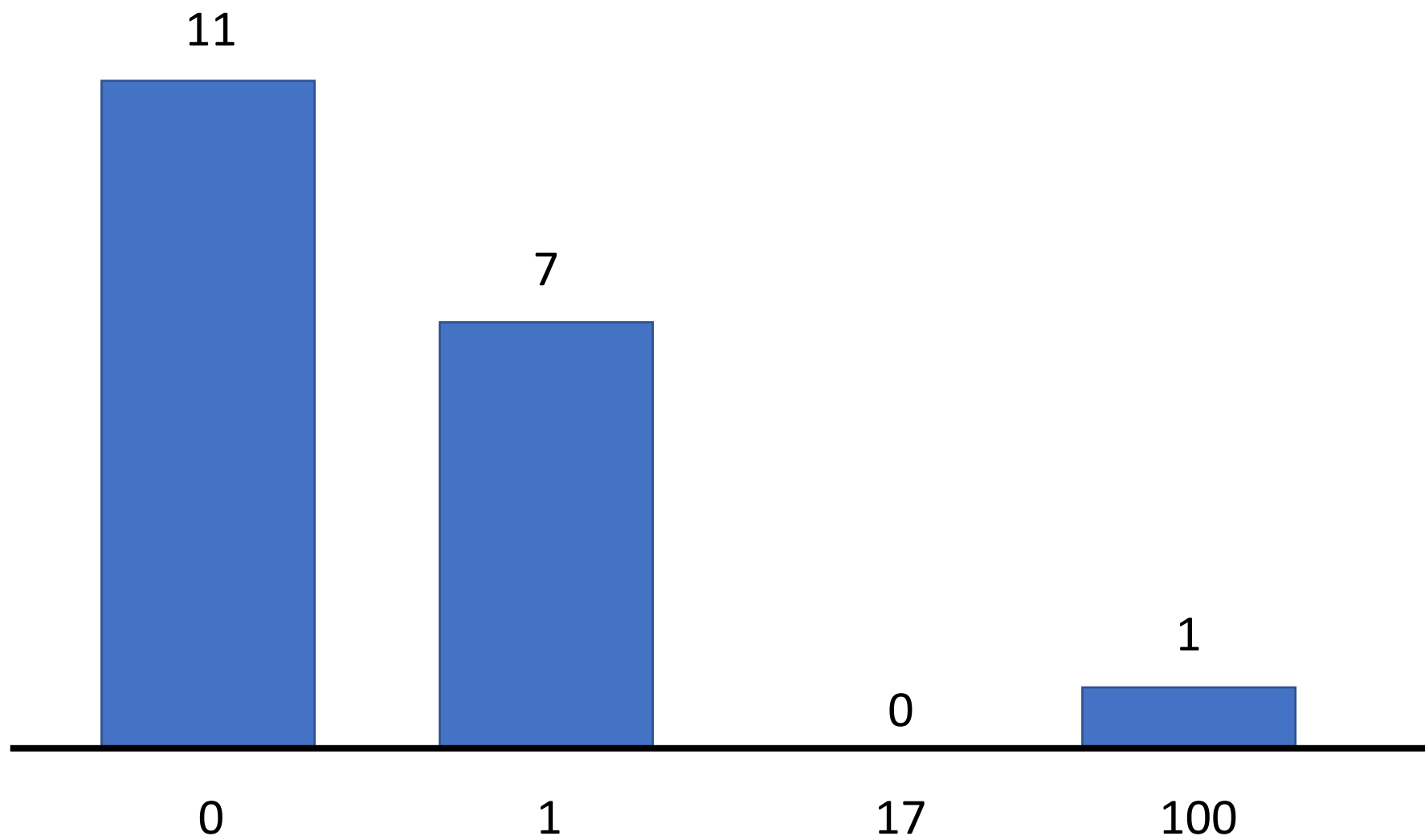
- 両方向に辺がある頂点を連結成分としてUnion-findで管理
- 異なる連結成分間の辺を別にsetとかで管理
- 毎日辺を増やしたり連結成分をつなげたりする
- 上手に実装すると  $O(N^2 \log N)$  や  $O(N^2)$
- 17点が取れる

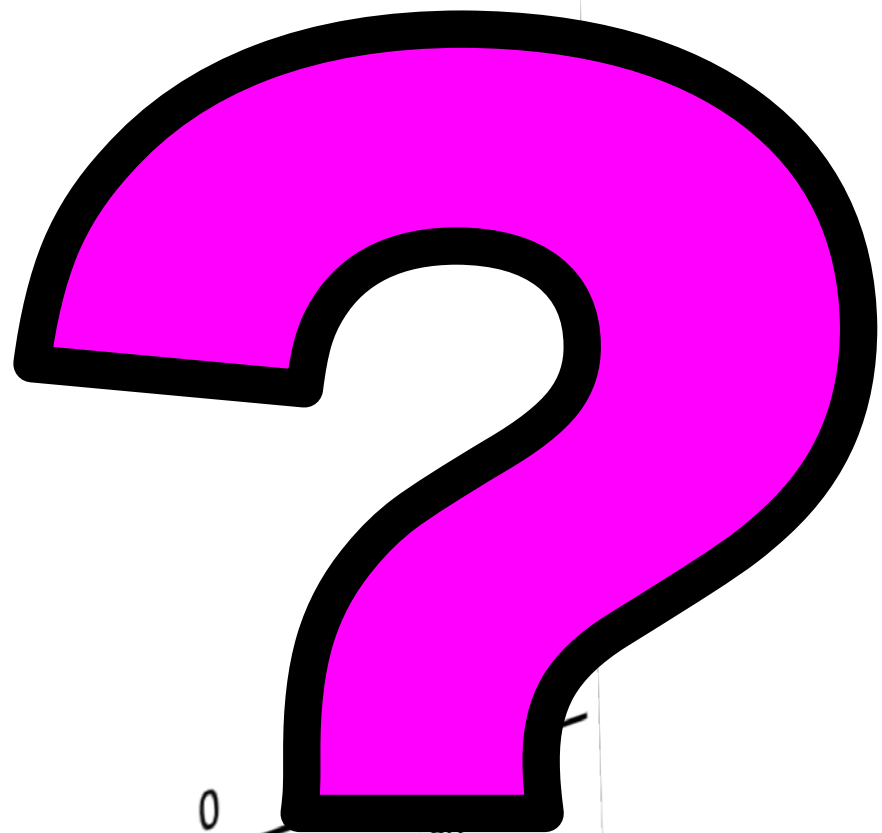
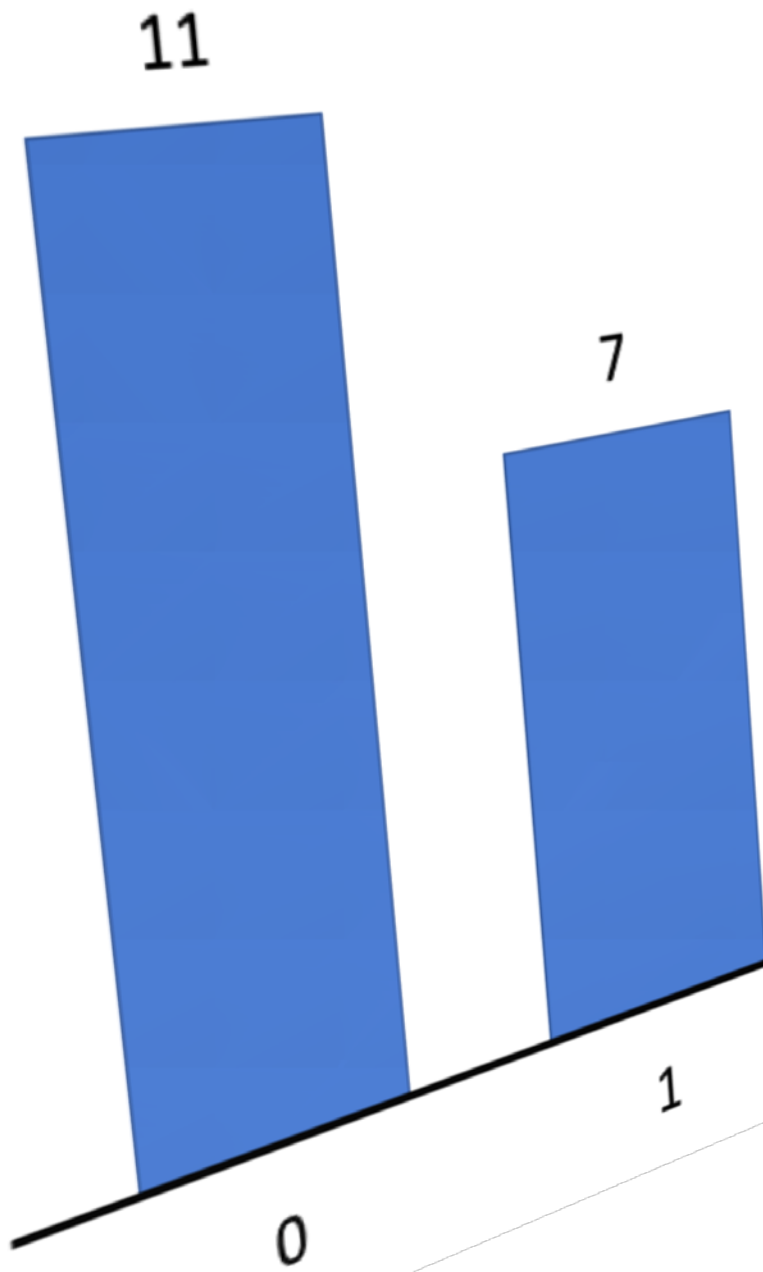
# 小課題3

- 要は連結成分をマージしてるだけ
- 連結成分が大きい方に小さい方をマージすると計算量が落ちる
- 上手に実装すると  $O(N \log^2 N)$
- 100点が取れる



得点分布





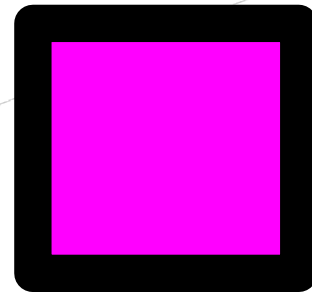
0

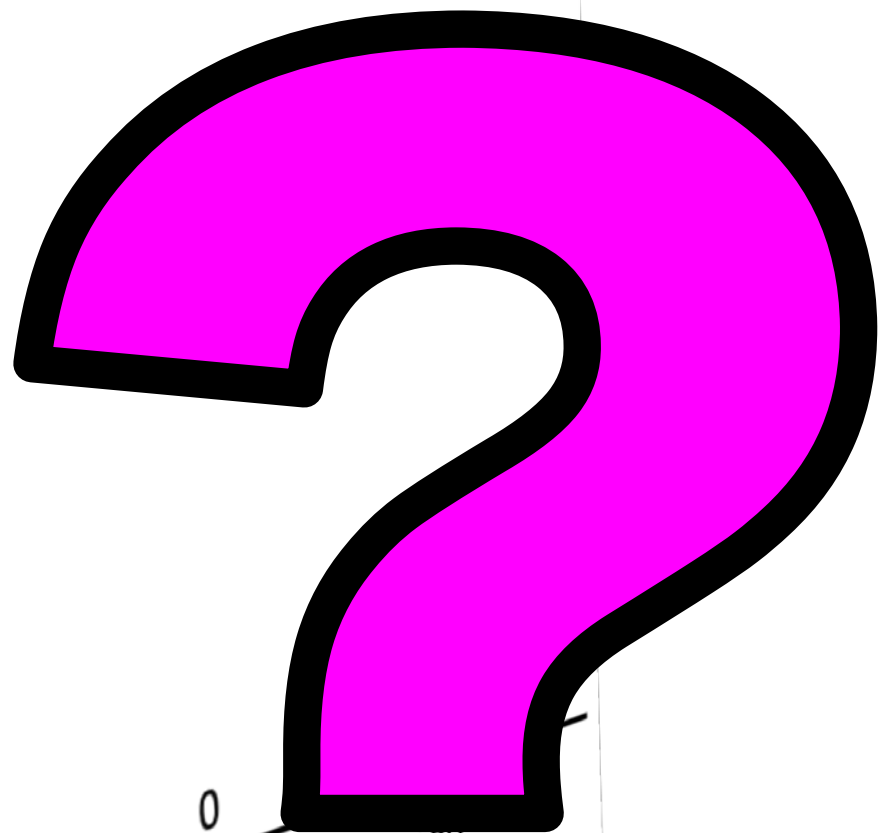
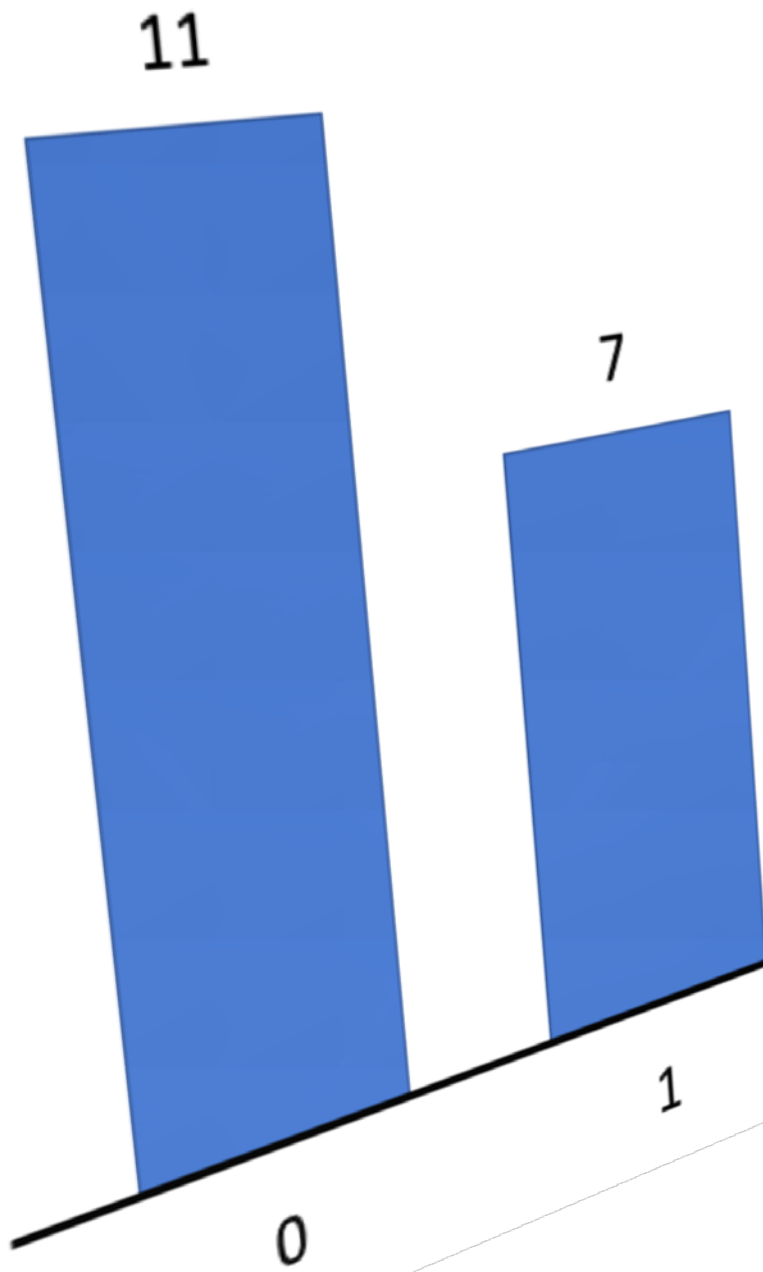
100

17

1

0





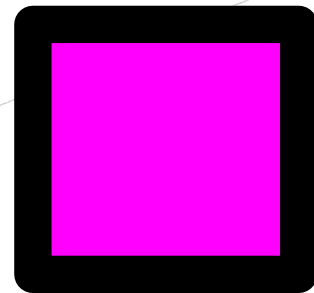
0

100

17

1

0



# 小課題3

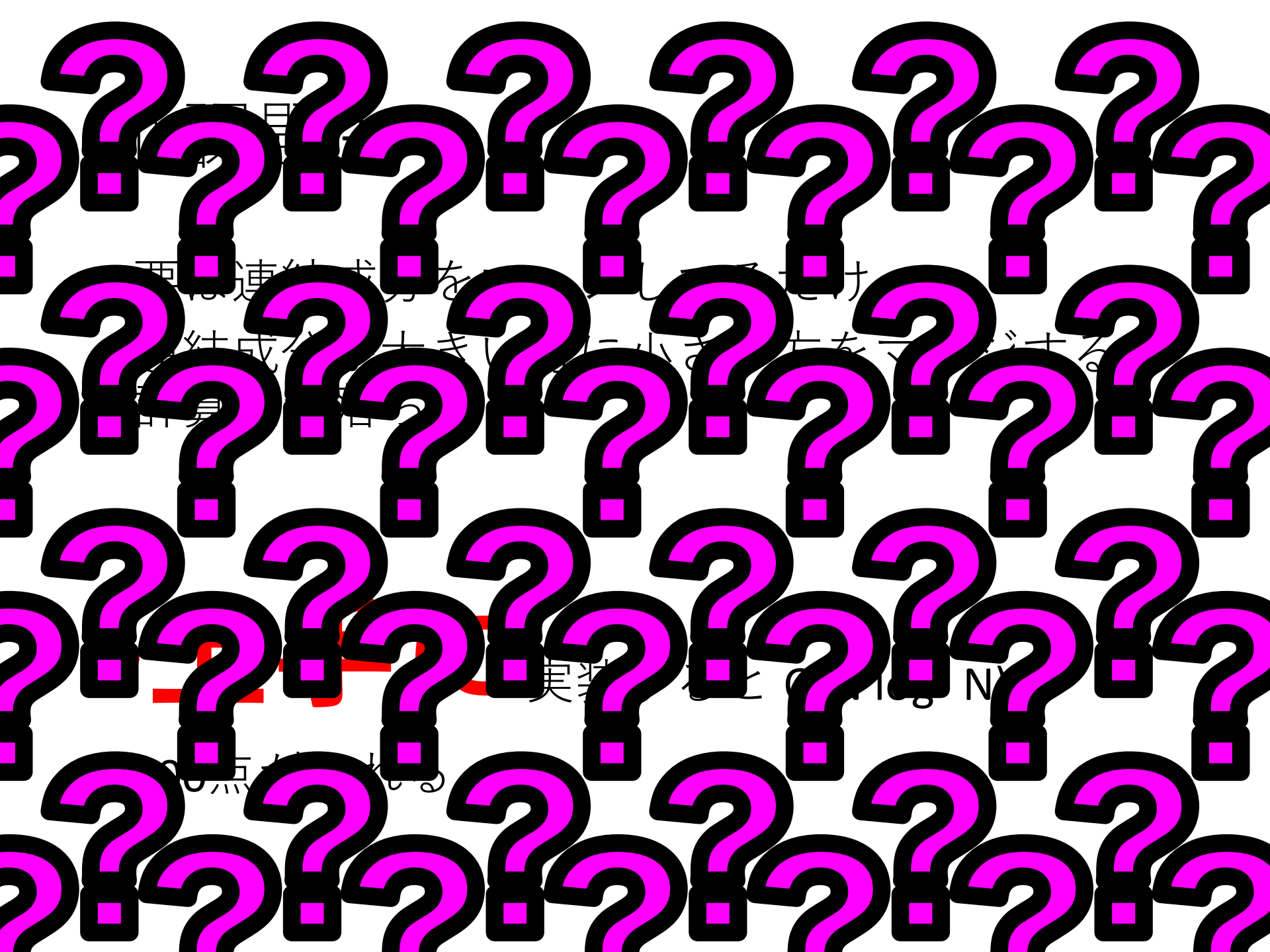
- 要は連結成分をマージしてるだけ
- 連結成分が大きい方に小さい方をマージすると計算量が落ちる
- 上手に実装すると  $O(N \log^2 N)$
- 100点が取れる

# 小課題3

- 要は連結成分をマージしてるだけ
- 連結成分が大きい方に小さい方をマージすると計算量が落ちる

• **上手に** 実装すると  $O(N \log^2 N)$

- 100点が取れる



# 100点が取れる例

- JOIのサイトにある解答例で100点が取れます
- けれども多分AtCoder上にある提出で綺麗なやつとかソースコード長が短いやつを見る方が勉強になると思う
- チューターのnxteruさんの実装が綺麗でわかりやすいと話題に  
<https://atcoder.jp/contests/joisc2020/submissions/16220023>



# **Ruins 3**

解説：岸田陸玖

# 問題概要

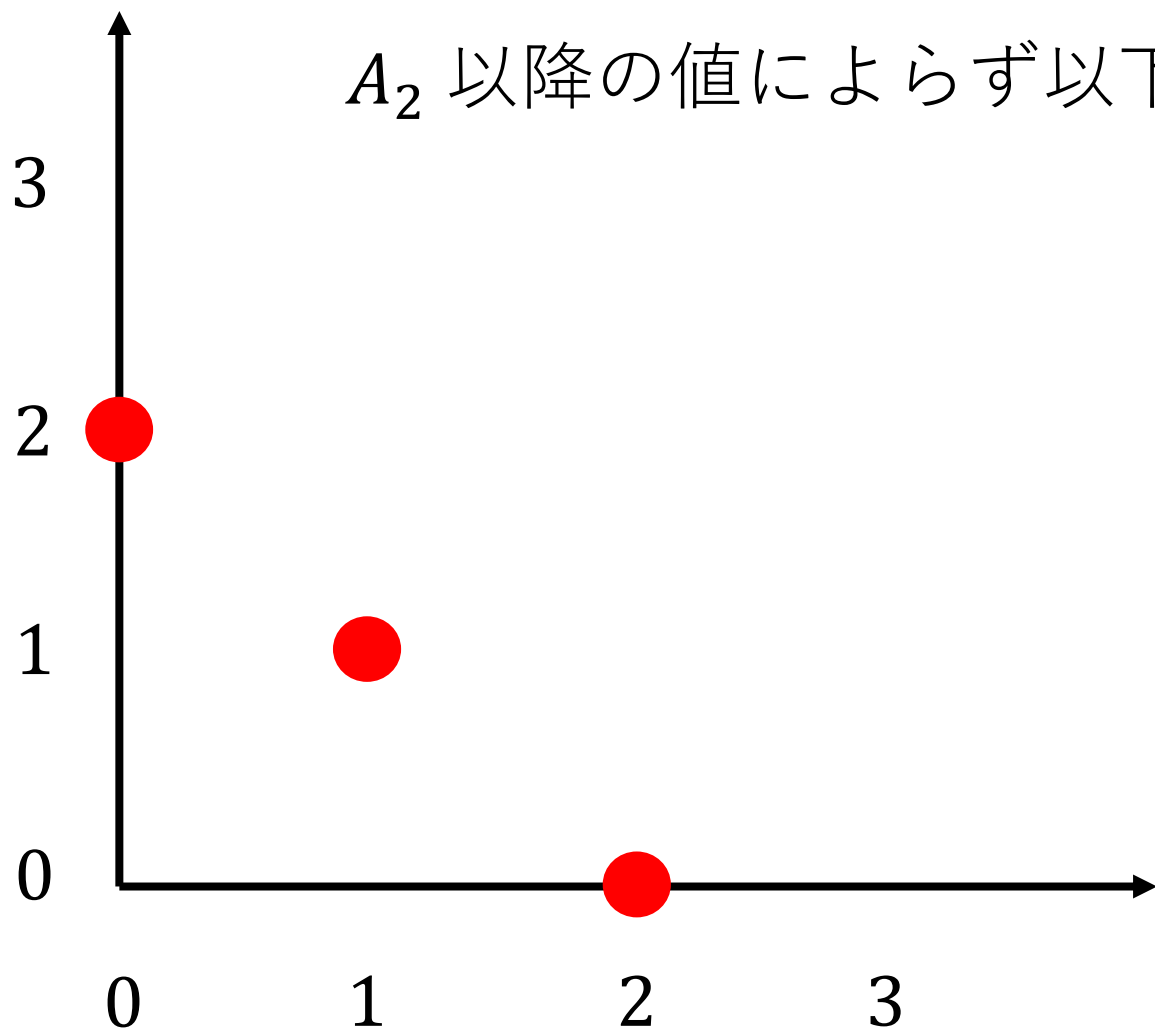
- $A$  は  $(1, 1, 2, 2, \dots, N, N)$  の順列であり、以下の操作を  $N$  回行った後の数列の正の値を持つ番号が指定される（ $i$  番目が指定されている番号であるとき、 $A_i$  は**残る**と呼ぶことにする）
- 2 個の番号に同じ値を持つ場合、番号が小さいほうを  $-1$  加算する
- $A$  の通り数は？

# 小課題 1 ( $N \leq 13$ )

- $(1, 1, 2, 2, \dots, N, N)$  の順列は  $\frac{(2N)!}{2^N}$  個ある
- $N = 13$  のとき約  $5 \times 10^{22}$  個あるので全探索では間に合わない
- 操作を繰り返し行うことで数列がどのように変化するかを考察してみる

$A_1 = 2$  のとき操作を行うことで  
どのような遷移で変化するのか考えてみる

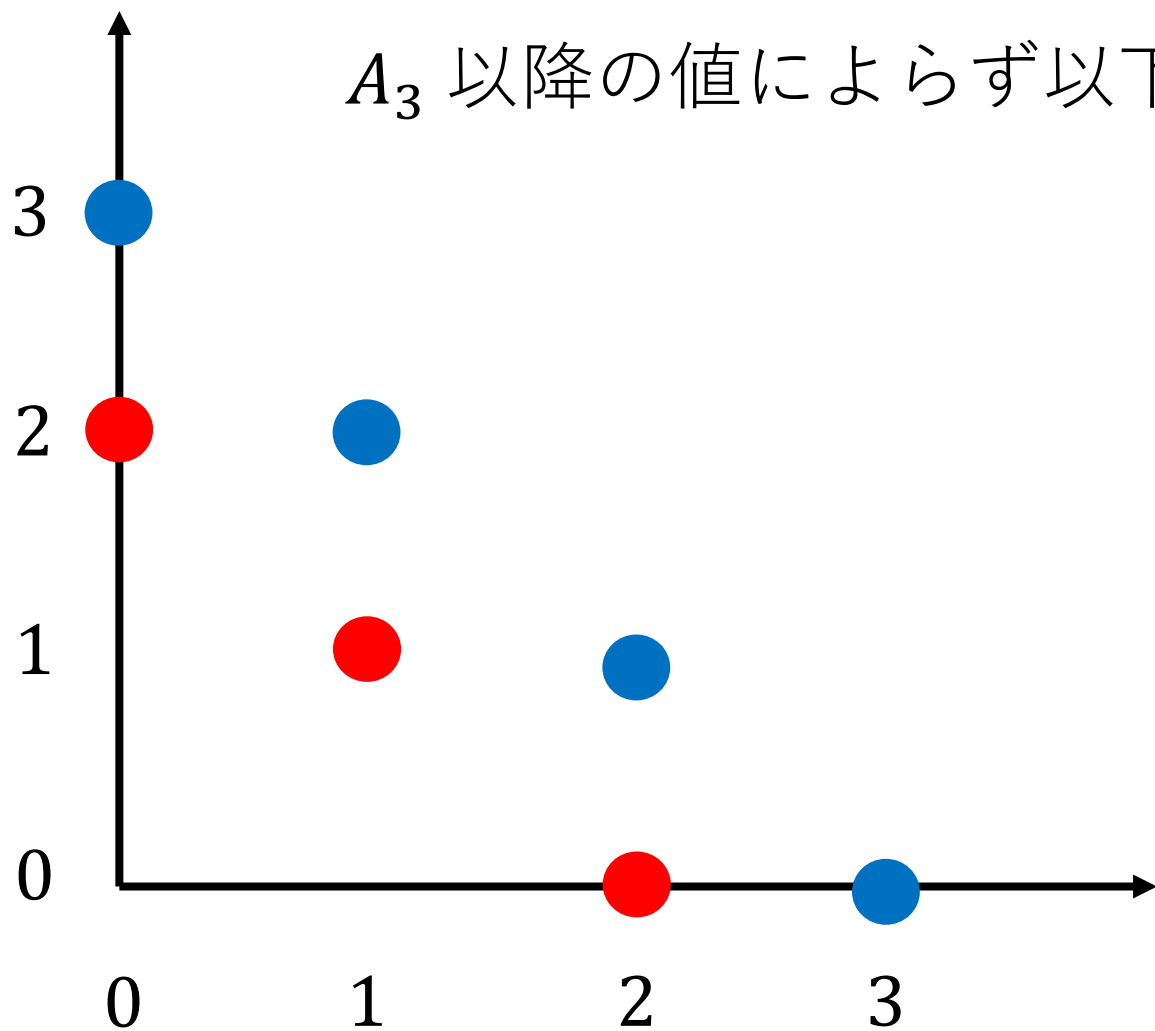
$A_2$  以降の値によらず以下のように遷移する



操作回数  
( $N = 3$ )

$A_2 = 3$  のとき操作を行うことで  
どのような遷移で変化するのか考えてみる

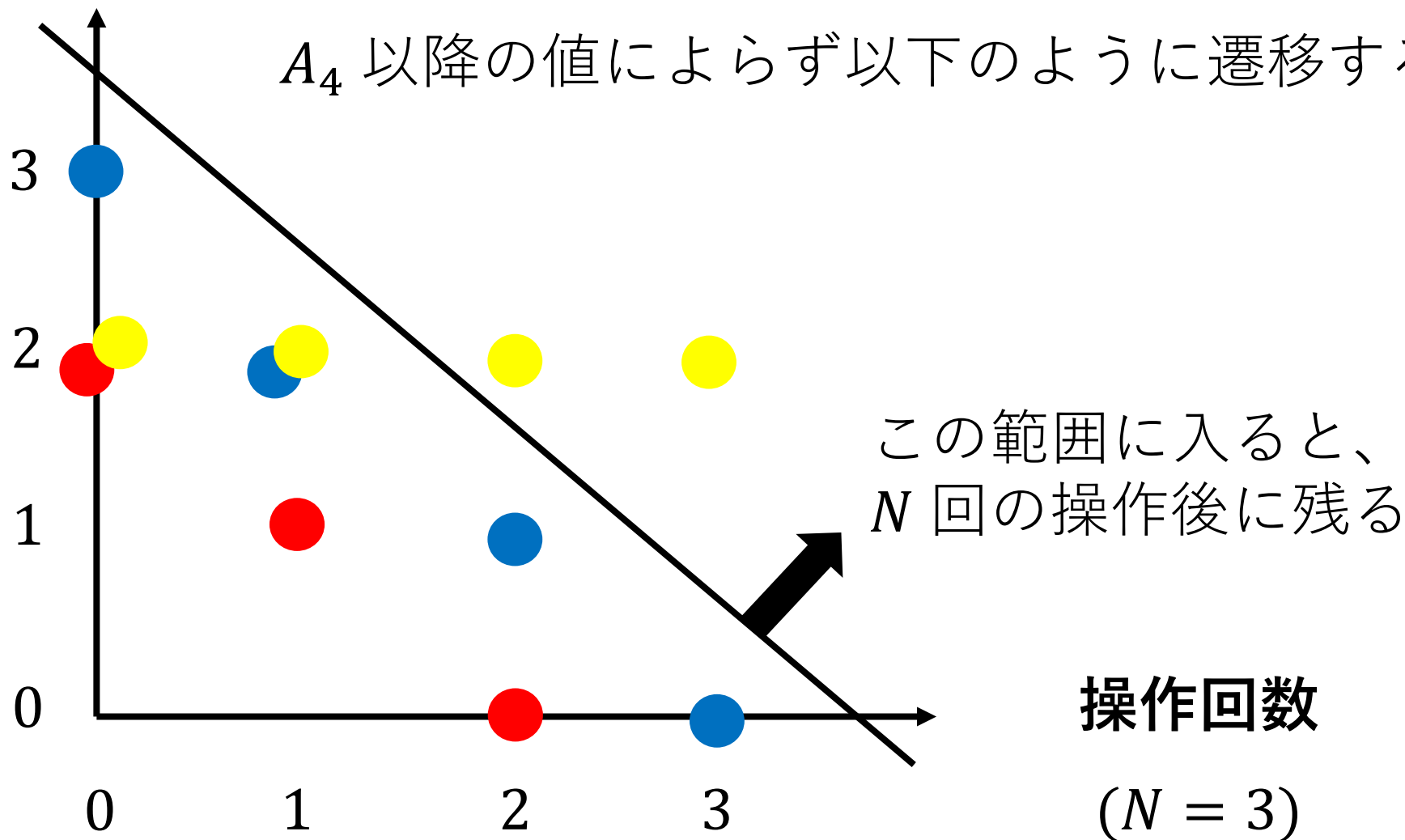
$A_3$  以降の値によらず以下のように遷移する



操作回数  
( $N = 3$ )

$A_3 = 2$  のとき操作を行うことで  
どのような遷移で変化するのか考えてみる

$A_4$  以降の値によらず以下のように遷移する



# 小課題 1 ( $N \leq 13$ )

- 前の例のように  $i$  番目の値が残るか否かは  $[1, i)$  番目の値にのみ依存することが推測できる
- 実際この推測は正しく、 $i$  番目の値が残る条件は次のように言い換えることができる
- $B_j = \#\{k \in [1, i) | A_k = j\}$  と定義する  
 $\sum_{j=k}^l (B_j - 1) \geq 0$  ( $k \leq l \leq N$ ) が成り立つような  $k \leq A_i$  が存在する

# 小課題 1 ( $N \leq 13$ )

- $B_j$  の値をkeyとして、1番から順に値を決めるようにDPをすればよい
- $0 \leq B_j \leq 2$  より  $O(3^N N)$  または  $O(3^N N^2)$  で解くことができる
- 6点！



## 小課題 2 ( $N \leq 60$ )

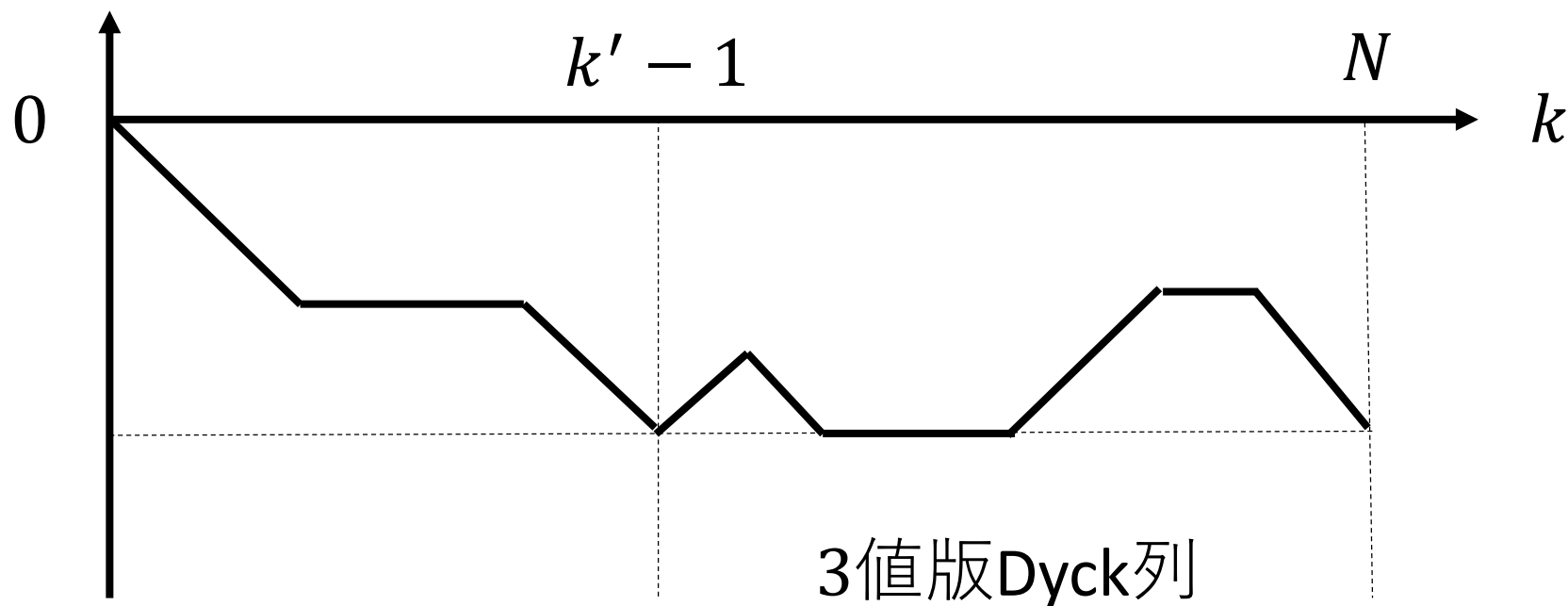
- $B_j$  の値すべてをkeyとしてDPするのはできない
- $\sum_{j=k}^l (B_j - 1) \geq 0$  ( $k \leq l \leq N$ ) が成り立つような  $k$  の最小値  $k'$  をkeyとしてDPできそう

# 小課題 2 ( $N \leq 60$ )

- 条件を次のように言い換えてみる
- $B_j = \#\{k \in [1, i) \mid A_k = j, A_k \text{は残らない}\}$  として  
 $\sum_{j=k}^l (B_j - 1) \geq 0$  ( $k \leq l \leq N$ ) が成り立つような  
 $k \leq A_i$  が存在する
- このように  $B_j$  の定義を変えてみると  
 $\sum_{j=k}^N (B_j - 1) = 0$  が成り立ち、**Dyck**列 (対応の  
取れた括弧列) に類似したものになる

## 小課題 2 ( $N \leq 60$ )

- $\sum_{j=1}^k (B_j - 1)$  の値の変化について考えてみる



## 小課題 2 ( $N \leq 60$ )

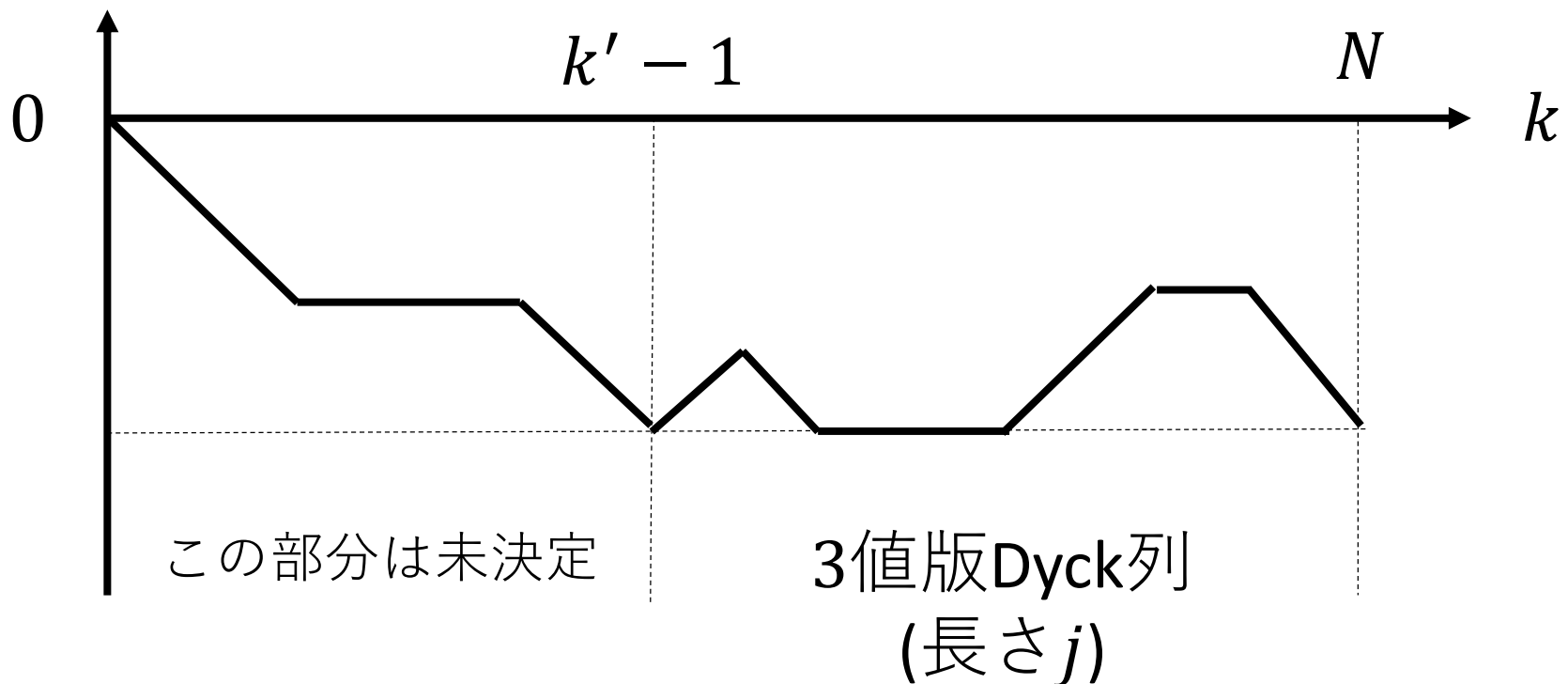
- $dp[i][j]$  を  $[1, i)$  番目まで  $A_i$  を（一部を除いて）決めた時、 $k' = N - j + 1$  となるような通り数と定義する

※残らない番号で3値版Dyck列に含まないのは未決定にしておく

- $dp[0][j] = \begin{cases} 1 & (j = 0) \\ 0 & (j \neq 0) \end{cases}$
- $dp[2N][N]$  が答え

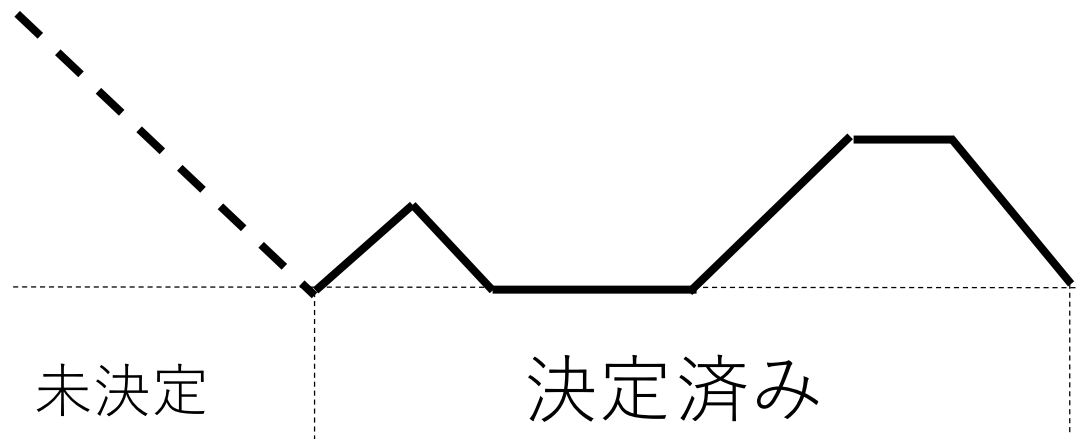
## 小課題 2 ( $N \leq 60$ )

- $\sum_{j=1}^k (B_j - 1)$  の値の変化について考えてみる



# 小課題 2 ( $N \leq 60$ )

- $A_i$  が残るときのDPの遷移について考察する
- 明らかに  $A_i \in [N - j + 1, N]$  であり、 $k'$  の値は変化しない
- $[N - j + 1, N]$  内でまだ置けることのできる値の個数を覚える必要がある



## 小課題 2 ( $N \leq 60$ )

- $A$  を  $(1, 1', 2, 2', \dots, N, N')$  の順列とする
- $1$  と  $1'$  は値としては同じように扱うが意味的には**異なる**ものとして扱う
- $dp[2N][N] \times \frac{1}{2^N}$  が答え
- $[N - j + 1, N]$  内でまだ置けることのできる値の個数は  $j - \#\{k \in [0, i) \mid A_k \text{ は残る}\}$
- このように、数える対象を変えることで計算が容易になる場合がある

## 小課題 2 ( $N \leq 60$ )

- $A_i$  が残るときのDPの遷移は以下のようなになる

$$dp[i][j] = dp[i-1][j] \cdot (j - \#\{k \in [0, i) \mid A_k \text{ は残る}\})$$



## 小課題 2 ( $N \leq 60$ )

- $A_i$  が残らないときのDPの遷移について考える
- $k'$  の値は変化しないとき、 $A_i \in [1, N - j + 1)$  なので未決定にしておく
- $k'$  の値は変化する場合、未決定にしていた番号の値を決めなければならない
- 未決定にしていた番号は  
 $num = \#\{k \in [0, i) \mid A_k \text{ は残らない}\} - j$  個存在する

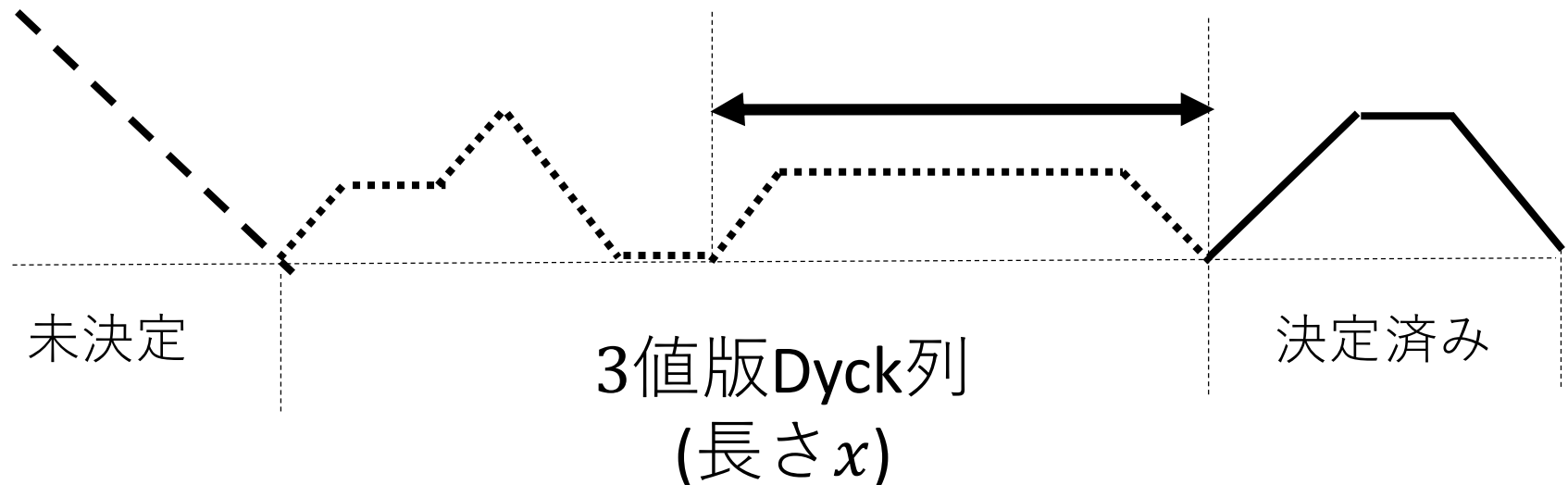
## 小課題 2 ( $N \leq 60$ )

- $k'$  の値が  $x$  だけ減ったとして、  
以下のように3値版Dyck列を付け加えたとする



## 小課題 2 ( $N \leq 60$ )

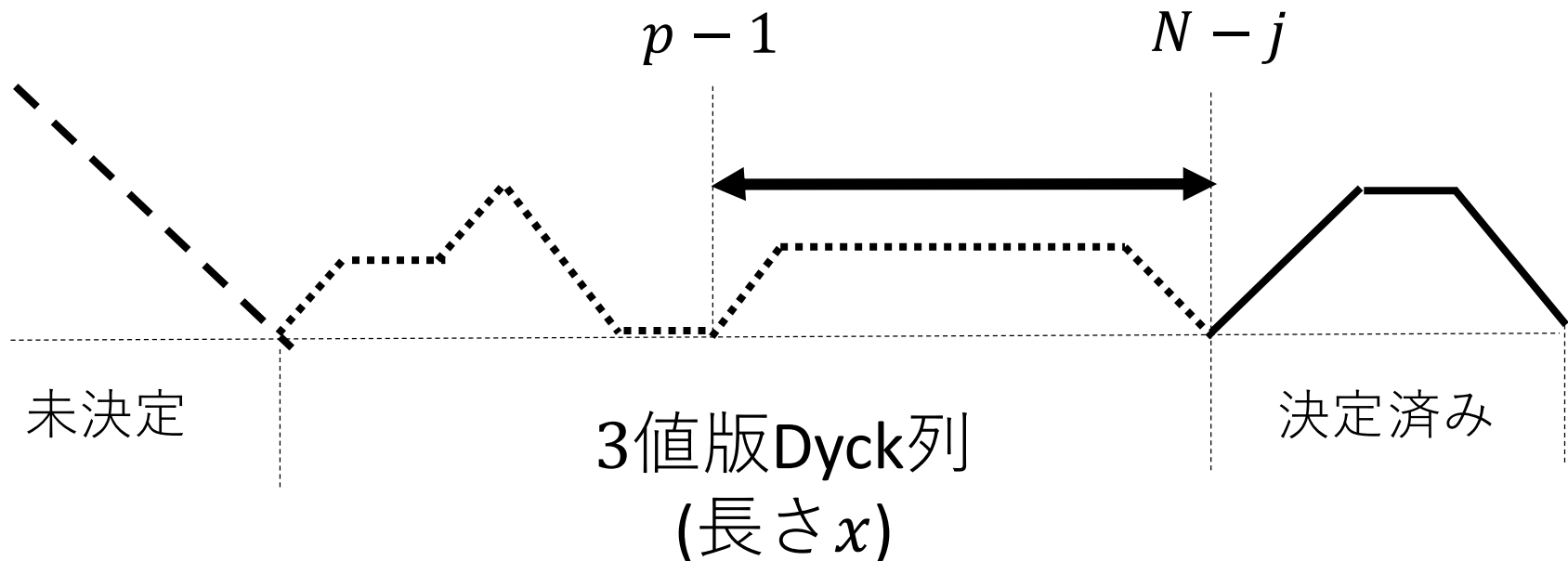
- このとき、 $A_i$  の値は端点のみ横線に接するような3値版Dyck列の最も右側になる



# 小課題 2 ( $N \leq 60$ )

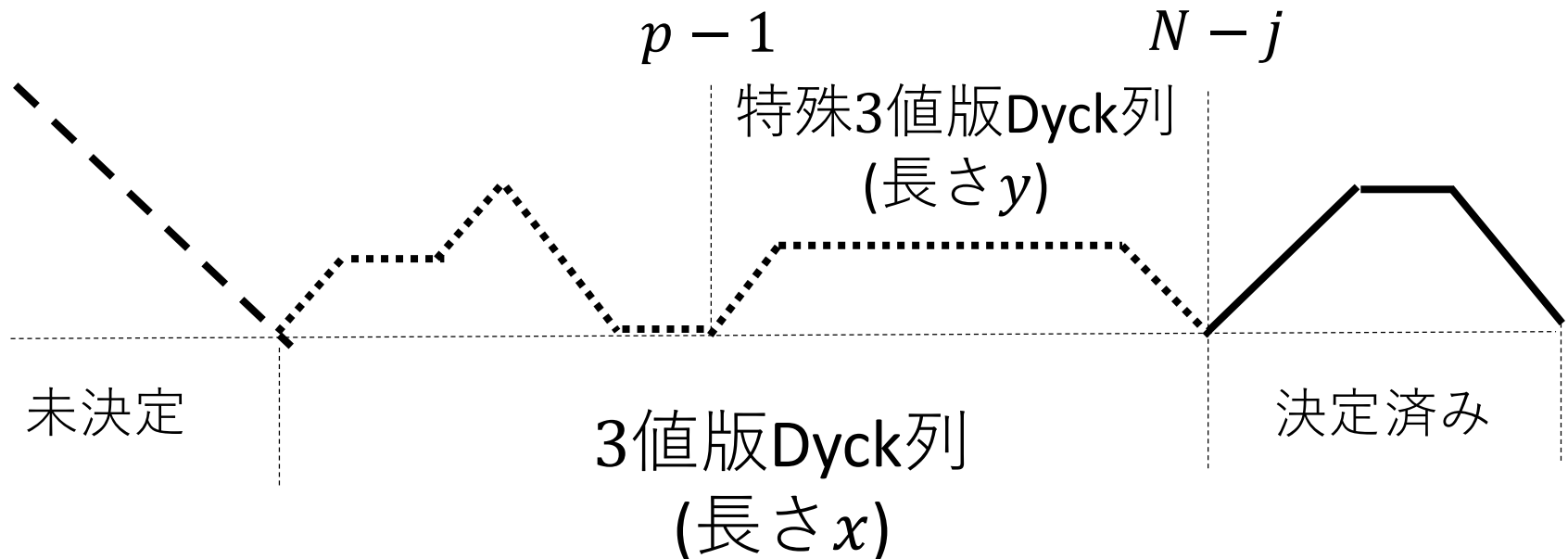
- 厳密に定義すると次のようになる

$p = \max\{k \in [N - j + 1 - x, N - j + 1) \mid \sum_{l=k}^{N-j} (B_l - 1) = 0\}$   
とすると、 $A_i \in [p, N - j + 1)$  になる



## 小課題 2 ( $N \leq 60$ )

- $y = N - j + 1 - p$  とおくと、 $A_i$  の値としてありえる値の個数は  $y$  個となる
- 端点のみ横線に接する3値版Dyck列を**特殊3値版Dyck列**と呼ぶことにする



## 小課題 2 ( $N \leq 60$ )

- 付け加える3値版Dyck列  $S = T + T'$   
（ $T'$  は特殊3値版Dyck列）が定まったとき、  
 $A$  の未決定である部分の決め方を考える
- 未決定である番号  $num + 1$  個（ $i$  番目も含む）  
の中で  $x$  個選んで番号の列  $I$  を決める  
ただし、 $i \in \{I_k | k \in [x - y + 1, x]\}$  が成り立つ  
とする
- このような列の取り方の通り数は  
$$\binom{num}{x-1} \cdot (x-1)! \cdot y$$

## 小課題 2 ( $N \leq 60$ )

- $i, i'$  を  $S_i + 1$  個だけ順に並べた数列を構成する
- 例えば  $S = (1, 0, -1)$  のとき  $(1, 1', 2)$   
または  $(1, 1', 2')$  となる
- $A_{I_j}$  の値はこの数列の  $j$  番目の値に対応させるようにする (値は適切にずらす)
- 数列の構成方法の通り数は
$$\prod_{i=1}^x (2 - |S_i|) = \prod_{i=1}^{x-y} (2 - |T_i|) \times \prod_{i=1}^y (2 - |T'_i|)$$

## 小課題 2 ( $N \leq 60$ )

- 長さ  $n$  の3値版Dyck列  $S$  に対して  $\prod_{i=1}^n (2 - |S_i|)$  の値を  $f(S)$  と定め、 $f(S)$  の総和を  $D_n$  とする
- 同様に特殊3値版Dyck列の場合を  $D'_n$  とする
- このように定義すると今までの考察からDPの遷移は以下ようになる

$$dp[i][j+x] += dp[i-1][j] \cdot \sum_{y=1}^x \binom{num}{x-1} \cdot (x-1)! \cdot y \cdot D_{x-y} \cdot D'_y$$

- $D_n, D'_n$  は  $O(N^2)$  のDPで計算できる



# 小課題 2 ( $N \leq 60$ )

- $A_i$  が残るとき

$$dp[i][j] = dp[i-1][j] \cdot (j - \#\{k \in [0, i) \mid A_k \text{ は残る}\})$$

- $A_i$  が残らないとき

$$dp[i][j] += dp[i-1][j]$$

$$dp[i][j+x] += dp[i-1][j] \cdot \binom{num}{x-1} \cdot (x-1)! \cdot \sum_{y=1}^x y \cdot D_{x-y} \cdot D'_y$$

- $O(N^4)$  で計算できる
- 58点！

# 小課題 3 ( $N \leq 600$ )

- $\sum_{y=1}^x y \cdot D_{x-y} \cdot D'_y$  の値をあらかじめ計算しておけばよい (OEISで調べてみると  $\binom{2x}{x}$  に等しいことがわかる)
- $O(N^3)$  で計算できる
- 100点！

# 小課題 3 ( $N \leq 600$ )

- 以下のように計算することもできる
- $X_i$  を  $i$  番目に小さい残る番号とする

$$dp'[0][j] = \begin{cases} 1 & (j = 0) \\ 0 & (j \neq 0) \end{cases}$$

$$dp'[i+1][j] += dp'[i][j] \cdot (j - i)$$

$$dp'[i+1][j+k] += dp'[i][j] \cdot \binom{X_{i+1} - i - j - 1}{k} \cdot \binom{2k}{k} \cdot k!$$

$$dp'[N][N] \times \frac{1}{2^N} \text{ が答え}$$

# 得点分布



6点



100点

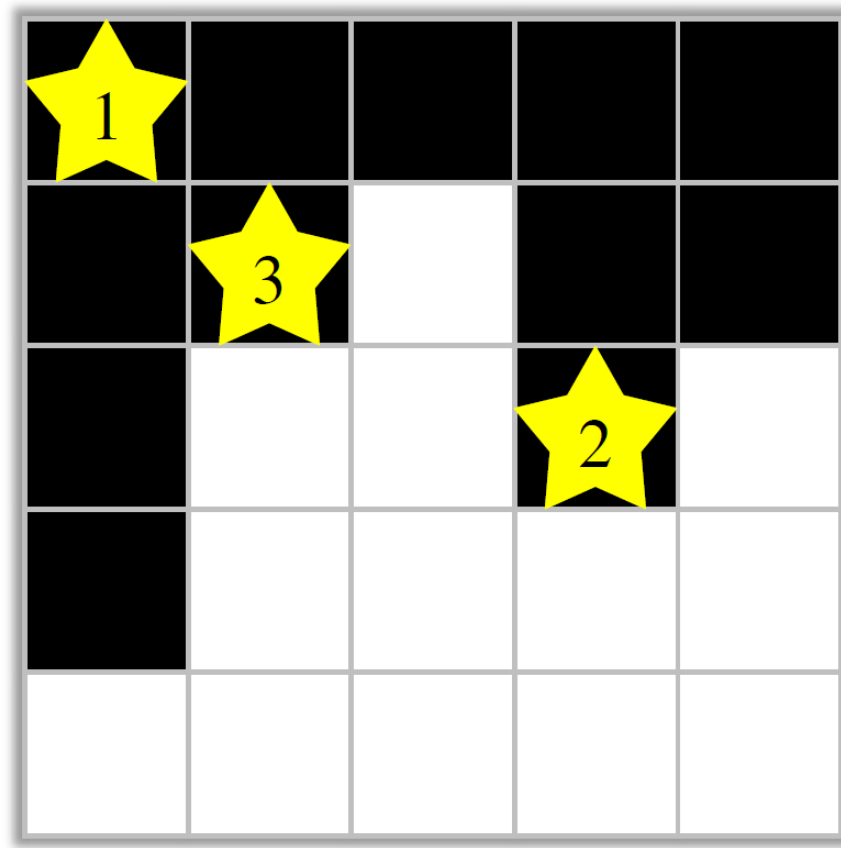
# Constellation 3

高谷 悠太

# 問題概要

- ビルの高さ  $A_1, \dots, A_N$ , 星  $(X_1, Y_1), \dots, (X_M, Y_M)$  が与えられる
- 各星を消すのにかかるコスト  $C_1, \dots, C_M$  が定まっている
- 以下を満たすように星をいくつか消すときの最小コスト
  - どの黒い長方形内にも高々1つしか星がない

# サンプル



# はじめに

- 星を消すのは面倒なので星を選ぶ問題にする
- どの黒い長方形内からも高々一つしか選べない



# Subtask 1

- 動的計画法をする
- $dp[l][r][h] := ([l, r] \times [0, h])$ から選べる最大コスト)
- 高さ $h$ の星を選ぶ or 高さ $h - 1$ に下がる
- 愚直にやると状態量3乗、遷移がたくさんで0点

# Subtask 1

- 実は状態数は少ない
- 遷移過程で出てくる  $(l, r)$  として考えられるものは  $N$  個しかない

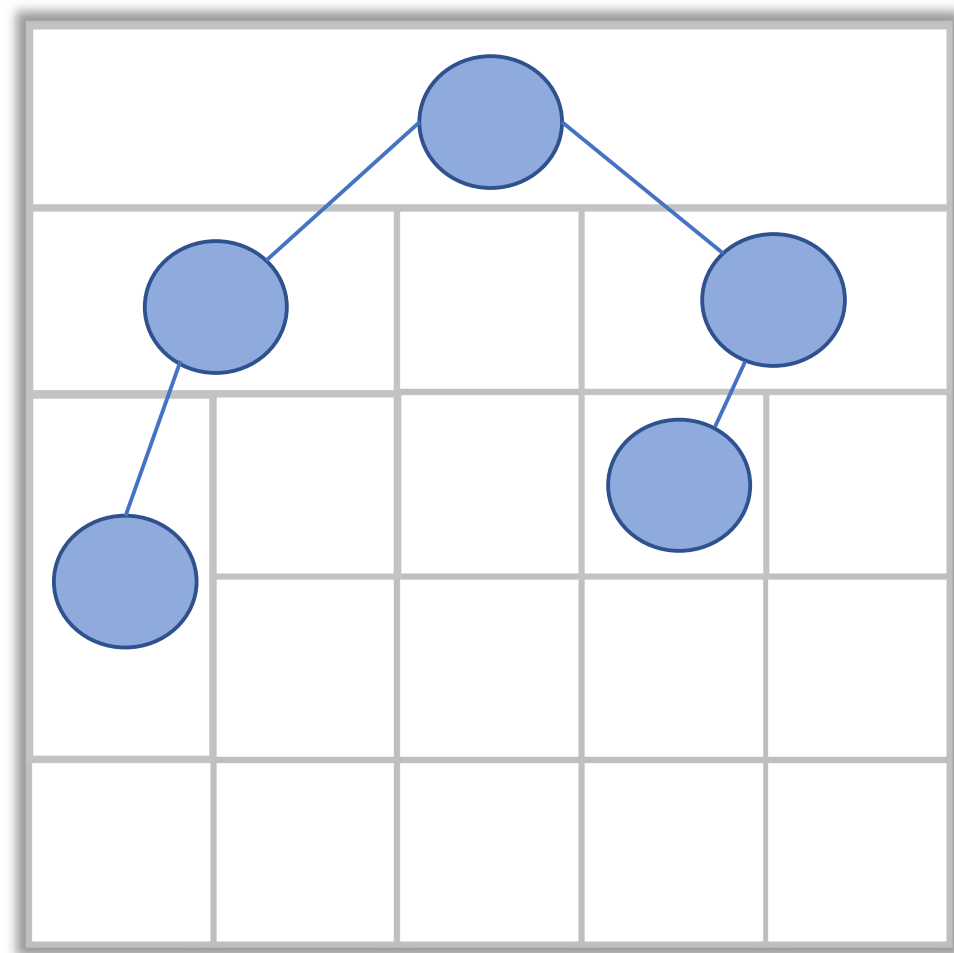
# Subtask 1

- 実は状態数は少ない
- 遷移過程で出てくる  $(l, r)$  として考えられるものは  $N$  個しかない
- Cartesian Tree の頂点に現れる区間だけだね

# Cartesian Tree の説明

- 最大値をとるindexを根にする
- 左右に分割する
- 左右で再帰的に木を作り、その根の親を全体の根につなげる

# Cartesian Tree の説明 – サンプル



# Subtask 2

- 必要な  $[l, r]$  の個数が  $N$  個だった
- 今度は  $h$  の候補が  $[l, r]$  に対して一意に定まる

# Subtask 2

- 必要な  $[l, r]$  の個数が  $N$  個だった
- 今度は  $h$  の候補が  $[l, r]$  に対して一意に定まる
  - $[l, r]$  に対応する Cartesian Tree の頂点に対応する高さ

# Subtask 2

- 必要な $[l, r]$ の個数が $N$ 個だった
- 今度は $h$ の候補が $[l, r]$ に対して一意に定まる
  - $[l, r]$ に対応するCartesian Treeの頂点に対応する高さ
- これでオーダーが一つ落ちる
  - 星についても、それが含まれる頂点でのみ遷移を行うことにする



# 満点

- 選べる星の集合の条件をちゃんと言い換える必要がある
- 「どの黒い長方形にも高々1つ」をCartesian Treeの言葉を用いて表す

# 満点

- まず各星には、それが含まれる頂点を対応させる
- その頂点は、星の $x$ 座標に対応する頂点の先祖にあたる

# 満点

- まず各星には、それが含まれる頂点を対応させる
- その頂点は、星の $x$ 座標に対応する頂点の先祖にあたる
- そこで、星に対応する頂点と $x$ 座標に対応する頂点を結ぶパスを考える

# 満点

- 星に対応する頂点と $x$ 座標に対応する頂点を結ぶパスを考える
- 「どの黒い長方形にも高々1つ」は  
「対応するパスがどの2つも交わらない」と言い換えられる

# 満点

- 結局次のような問題になる
  - 根方向のみに伸びるパスがいくつか与えられる
  - 各パスにはスコアが定まっている
  - どの2つも交わらないようにパスをいくつか選ぶ
  - 選んだパスのスコアの合計の最大値は？

# 満点

- 結局次のような問題になる
  - 根方向のみに伸びるパスがいくつか与えられる
  - 各パスにはスコアが定まっている
  - どの2つも交わらないようにパスをいくつか選ぶ
  - 選んだパスのスコアの合計の最大値は？
- これは木dpをすれば解ける

# 満点

- 具体的には、各頂点について、その頂点を根とする部分木に含まれるパスのみを考えて同じ問題を解く
- この答えをbottom-upの順に計算していく
- 子孫での答えが分かっているときに、根での答えを求めたい

# 満点

- まず、根を含むパスを選ばないとき
- この場合の答えは、直接の子どもでの答えを足し合わせたものになる
- 問題なのは、根を含むパスを選ぶ場合



# 満点

- 選ぶパスを1つ固定する
- そのときに得られるスコアの最大値はそのパスのスコアに

そのパスを除いて得られる根付き森における根での答えの総和  
を加えたもの

- これをデータ構造を用いて計算したい

# 満点

- 各頂点で、直接の子での答えの総和を計算しておく
- その値をパス上の頂点に対して足し合わせたものから、根を除いたパス上の頂点での答えを足し合わせたものの差が、先ほど求めたかった値となる

# 満点

- 各頂点で、直接の子での答えの総和を計算しておく
- その値をパス上の頂点に対して足し合わせたものから、根を除いたパス上の頂点での答えを足し合わせたものの差が、先ほど求めたかった値となる
- 結局パス上に書かれた値の総和を求める問題が解ければよい
- これは解けるね

# Harvest 解説

# 問題概要

- 円周上に人がいる
- りんごの木も植えてある
- 人間が等速度で時計回りに動いていく
- りんごの木は  $C$  秒経つと実をつける
- ある時刻までにある人間が取ったりんごの個数は？  
というクエリに答えていく

# 問題概要

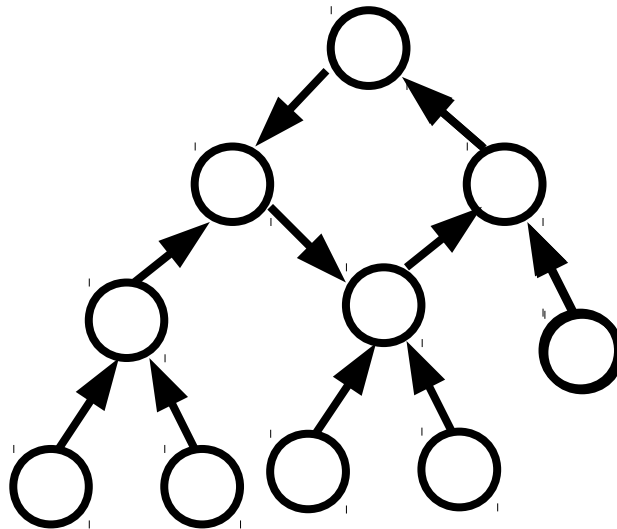
- 円周上に人がいる
- りんごの木も植えてある
- りんごが等速度で反時計回りに動いていく
- りんごの木は  $C$  秒経つと実をつける
- ある時刻までにある人間が取ったりんごの個数は？  
というクエリに答えていく

# subtask 1

- C 秒経つと新しい実をつける  
→ 人間  $x$  にりんごを渡した木が次にりんごを渡す人間は...  
→  $x$  から  $C$  以上反時計回りに進んだあと初めて出てくる人間
- $x$  の次に  $y$  に渡す, というときに,  $x \rightarrow y$  の辺を貼ったグラフを考える

# subtask 1

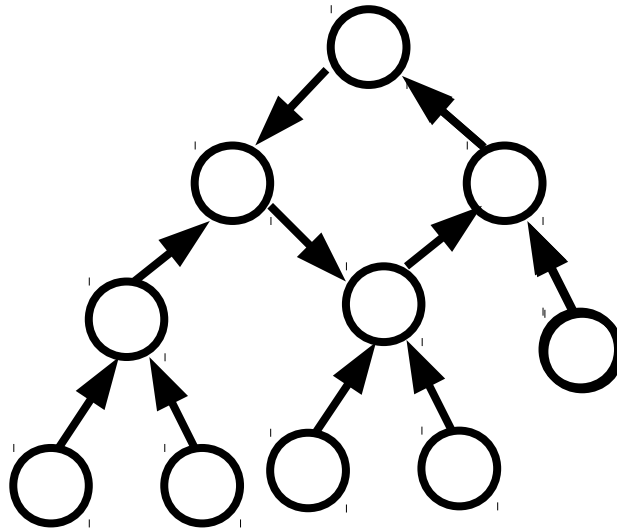
- 各頂点の出次数が 1 のグラフ  
→ サイクルと木が集まってできたグラフ





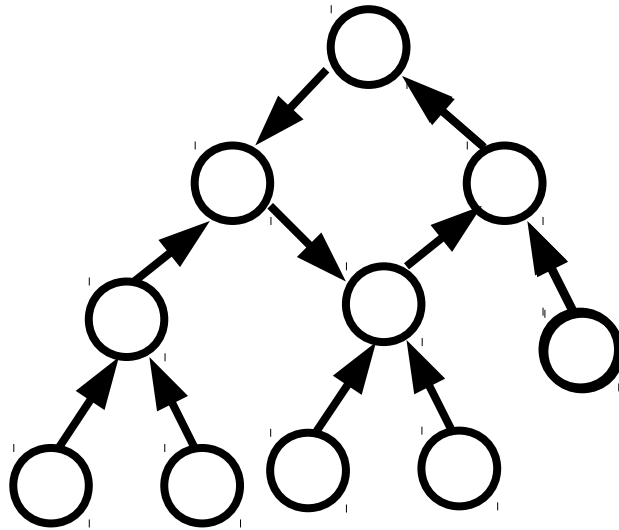
# subtask 1

- 各りんごの木は, 適当な頂点からスタートし, どんどんりんごを配っていく



# subtask 1

- 木の部分の頂点には, 一度しか配らない
- サイクルの部分の頂点には, 一定周期で配る



# subtask 1

- 木の部分の頂点に関するクエリ  
どのりんごが時刻いくつで届けられるか管理しておけばいい
- サイクルの部分の頂点に関するクエリ  
どのりんごが時刻いくつで最初に届けられるか管理しておけばいい
- $O(NM+QM)$  とか, なんでもいい

## subtask 2

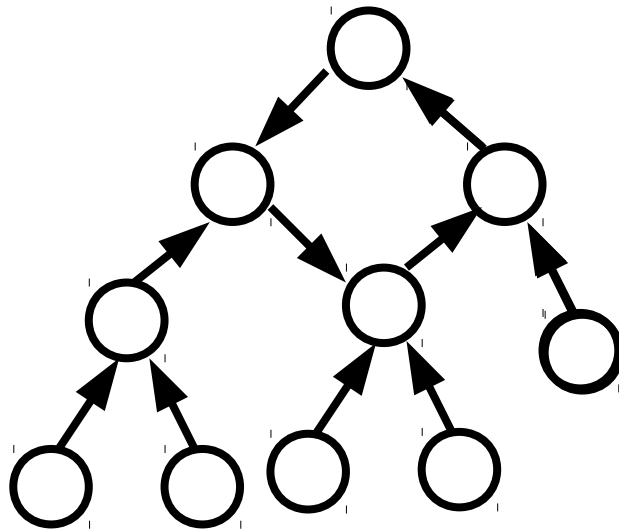
- 時刻がでかい
- どのりんごも到達できる頂点には一度以上到達してる

## subtask 2

- 木の部分の頂点に関するクエリ  
その頂点に到達するりんごの個数を数えればいい
- それは、木においてその頂点とその子孫からスタートするりんごの数が数えられればいい
- 最初に一回dfsするだけ

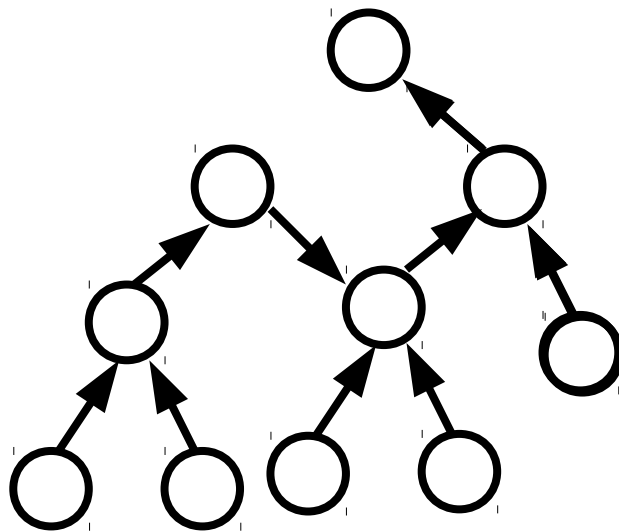
## subtask 2

- サイクルの部分に関するクエリ
- サイクルを適当な辺で切って, 木にしておく



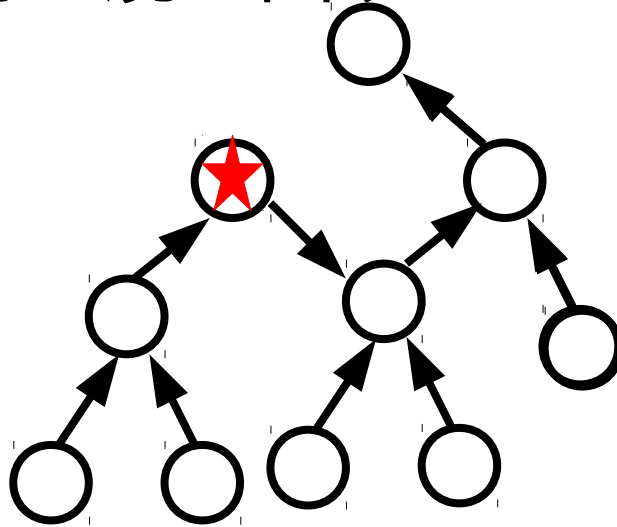
## subtask 2

- サイクルの部分に関するクエリ
- サイクルを適当な辺で切って, 木にしておく



## subtask 2

- そうすると木の問題になる
- 本来サイクルをループするりんごは, 切ったところから定期的に湧き出すりんごとして処理する.





## subtask 2

- 定期的に湧き出すりんごを何個獲得するかが分かればいい
- 湧き出してくる頂点を  $x$  とおく
- 頂点  $Y$  で時刻  $T$  までに得られるりんごの数  
→ 頂点  $X$  で時刻  $T - (x \text{ から } y \text{ へ 至るまでの時間})$  までに得られるりんごの個数

## subtask 2

- 結局, 頂点  $x$  に関するクエリだけ答えられればいい
- 頂点  $x$  からあるりんご  $r$  が湧き出すタイミングは,  
 $a_r + Dt$  ( $t=0,1,2,\dots$ ) という形なる ( $D$  はサイクルを一周するのにかかる時間で,  $r$  に寄らない定数)
- 時刻  $T$  のときのクエリの答えは  
 $\sum (T - a_r) / D + 1$  となる(割り算は切り捨て)

## subtask 2

- $a_r = b_r * D + c_r$
- $T = p * D + q$  とおく
- $c_r \leq q$  の場合  $(T - a_r) / D + 1 = p - b_r + 1$
- $c_r > q$  の場合  $(T - a_r) / D + 1 = p - b_r$
- よって,  $a_r$  を  $c_r$  の昇順にソートしておき, 適切に累積和を計算すれば, 一つの  $T$  に対しては  $O(\log M)$  で答えが求められる.
- 計算量は  $O((N+M)\log(N) + Q(\log M))$  とか

# subtask 3

- 木の頂点に関するクエリ
- その木の根を  $x$  とする, クエリで聞かれた頂点を  $y$  とする.
- 時刻  $T$  までに  $y$  に到達するりんごの個数=時刻  $T + (y$  から  $x$  に至るまでの時間) までに  $x$  に到達する,  $y$  に到達可能なりんごの個数

# subtask 3

- 木の頂点にオイラーツアーで順番をつけておく
- $y$  に到達可能なりんご → 始点となる頂点がオイラーツアー順である範囲に収まるりんご
- 各りんごに対し, (始点となる頂点のオイラーツアー順序) と, ( $x$  に到達する時刻) をプロットすると, 二次元平面の矩形内の点の個数をカウントする問題になる

## subtask 3

- クエリがオフラインなので, 平面走査+BIT でできる
- $O((M+Q)\log M)$

## subtask 3

- サイクルの頂点に関するクエリ
- 先ほどと同様, 定期的に湧き出すりんごを処理できればいい
- 求めたいものは,  $\sum \max((T-a_r)/D+1, 0)$

# subtask 3

- sub2 とほぼ同様にできる.
- 各りんごを,  $a_r$  の値と  $a_r \% D$  の値でプロットすると, 二次元平面の矩形領域にある点の重みの総和を数えることになる.
- 木の頂点と同様に, 平面走査+BIT で  $O((M+Q)\log M)$  でできる



## subtask 3

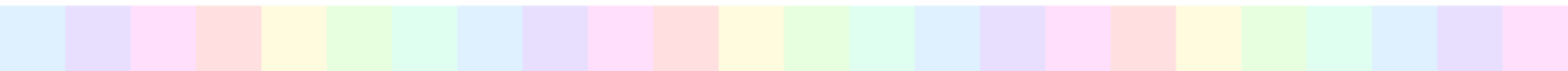
- 全部合わせて,  $O((N+M)\log N + (M+Q)\log M)$  でできる.



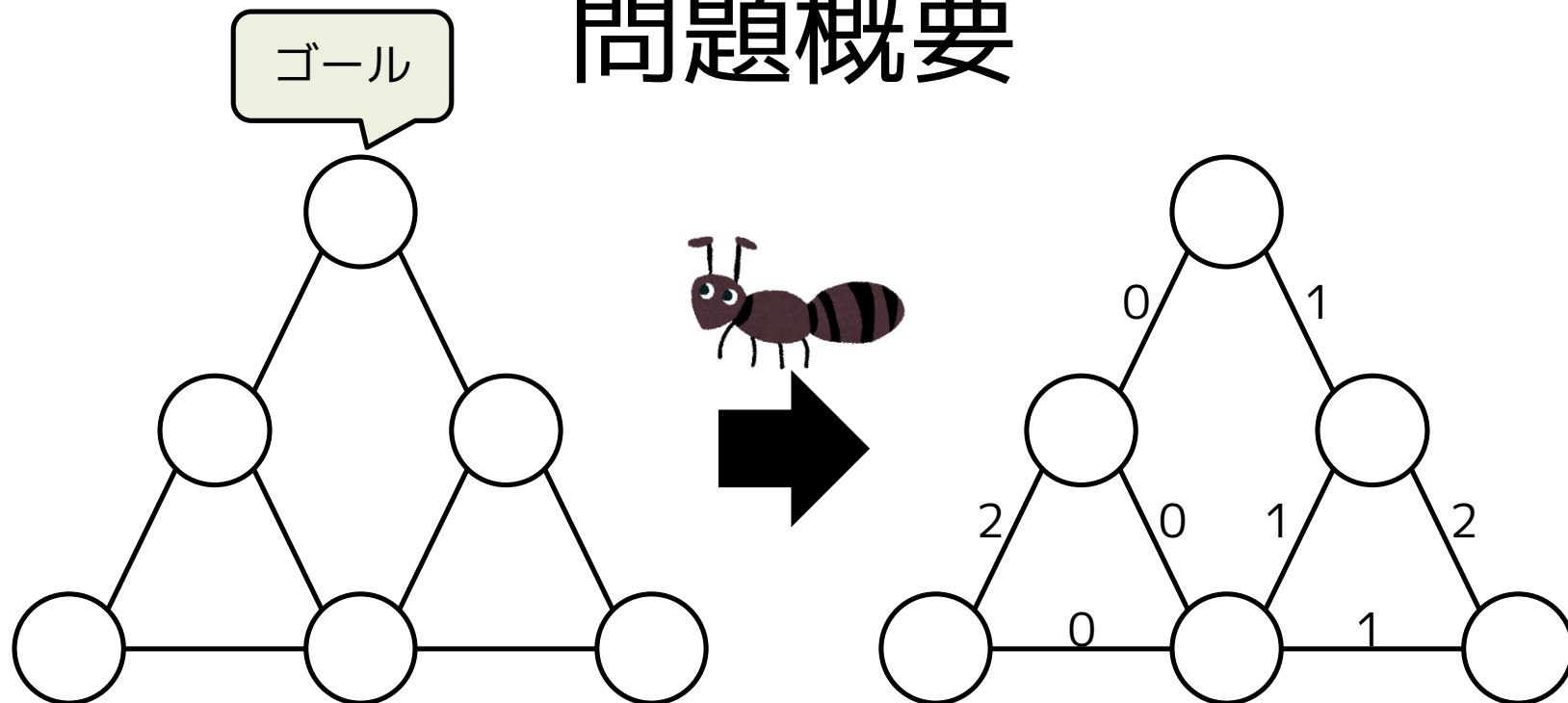
# 迷い猫 (Stray Cat)

JOI 春合宿 2020 Day 3

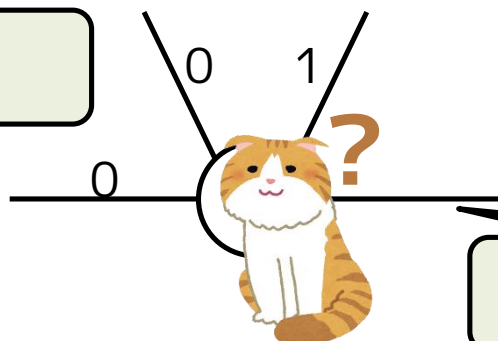
解説：保坂



# 問題概要



「0 が 2 本, 1 が 1 本」



「ここは直前に通った」

# 問題概要

- Anthony

- グラフをもらって辺に印を書く



- Catherine

- どこから頂点 0 に行きたい
- 辺は印でしか区別できない (直前に通った辺はわかる)



- 類題 (?) : Navigation (2015)

- 頂点に整数を書いて伝える

# 制約

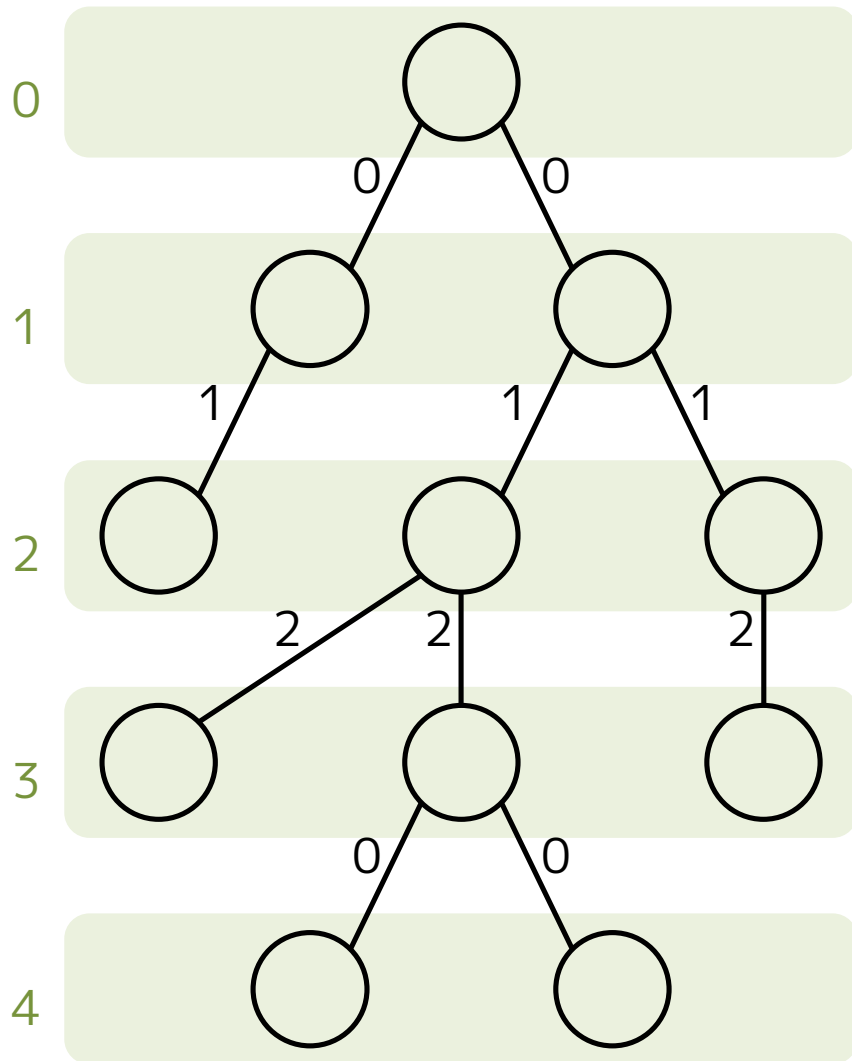
## 15 点

- 0, 1, 2 を書ける
- 木とは限らない
- 最短距離でやる

## 85 点

- 0, 1 を書ける
- 木
- 最短距離 + 6 まで OK

# 小課題 {1, 3} (4 点)

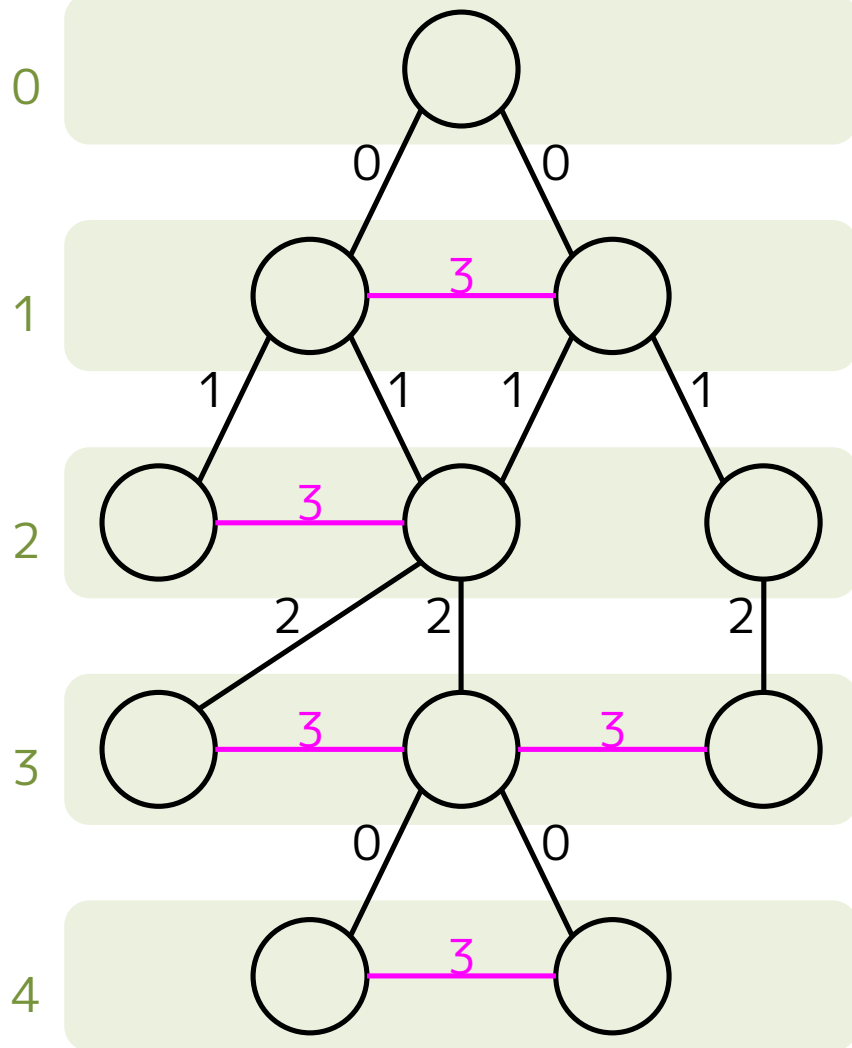


- 距離  $d$  と距離  $d+1$  を結ぶ辺に  $d \bmod 3$  を書く



- 根に向かう唯一つの頂点がわかる

# 小課題 {1, 2} (4 点)



- 距離  $d$  と距離  $d+1$  を結ぶ辺に  $d \bmod 3$  を書く
- 距離  $d$  と距離  $d$  を結ぶ辺に 3 を書く



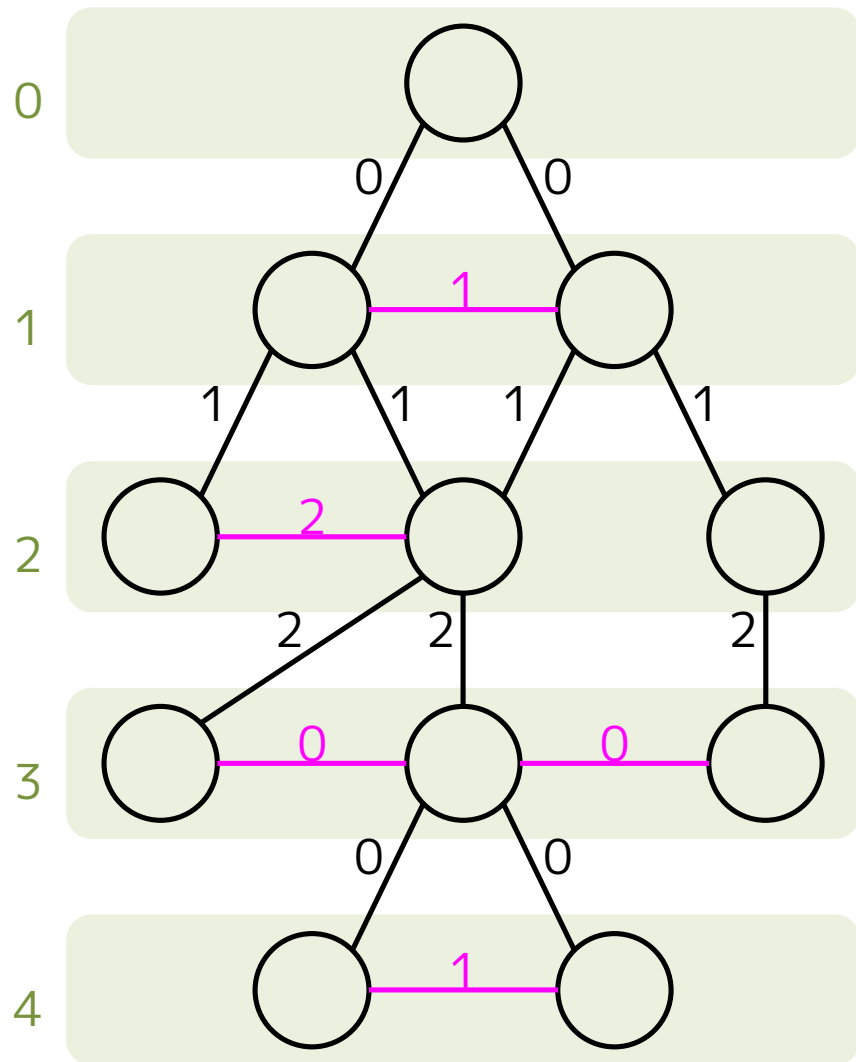
- 根方向がわかる

# BFS DAG

- BFS で頂点 0 からの距離を求めると各辺は以下のいずれか
  - 距離  $d$  と距離  $d + 1$  を結ぶ
    - そうというようなものだけ集めると木とは限らない DAG になる
  - 距離  $d$  と距離  $d$  を結ぶ



# 小課題 {1, 2, 3, 4} (15 点)

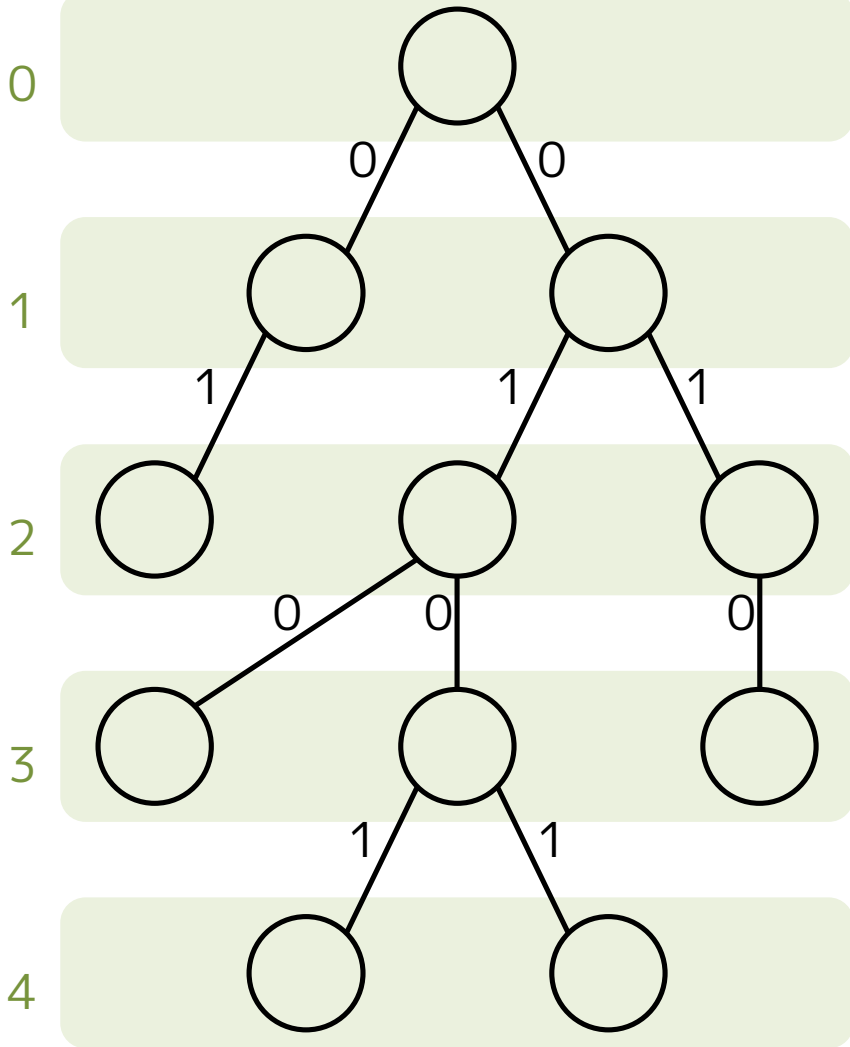


- 距離  $d$  と距離  $d+1$  を結ぶ辺に  $d \bmod 3$  を書く
- 距離  $d$  と距離  $d$  を結ぶ辺に  $d \bmod 3$  を書く



- 根方向がわかる

## 小課題 {5} (5 点)



- 距離  $d$  と距離  $d+1$  を結ぶ辺に  $d \bmod 2$  を書く



- 最初適当に進む
- そのうち葉に着いて以降根方向がわかる

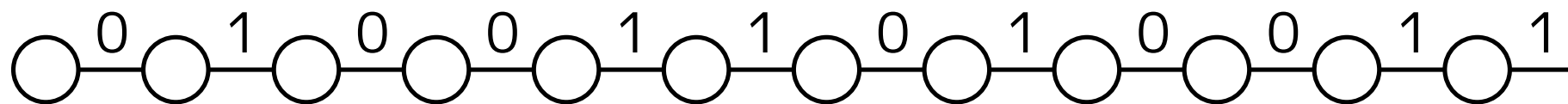
# 考察

- 次数 2 の頂点に降り立ったときやばそう
  - ◻ 「1 本だけ異なる」ようにできない
- 木の問題なので特殊な木を考えましょう

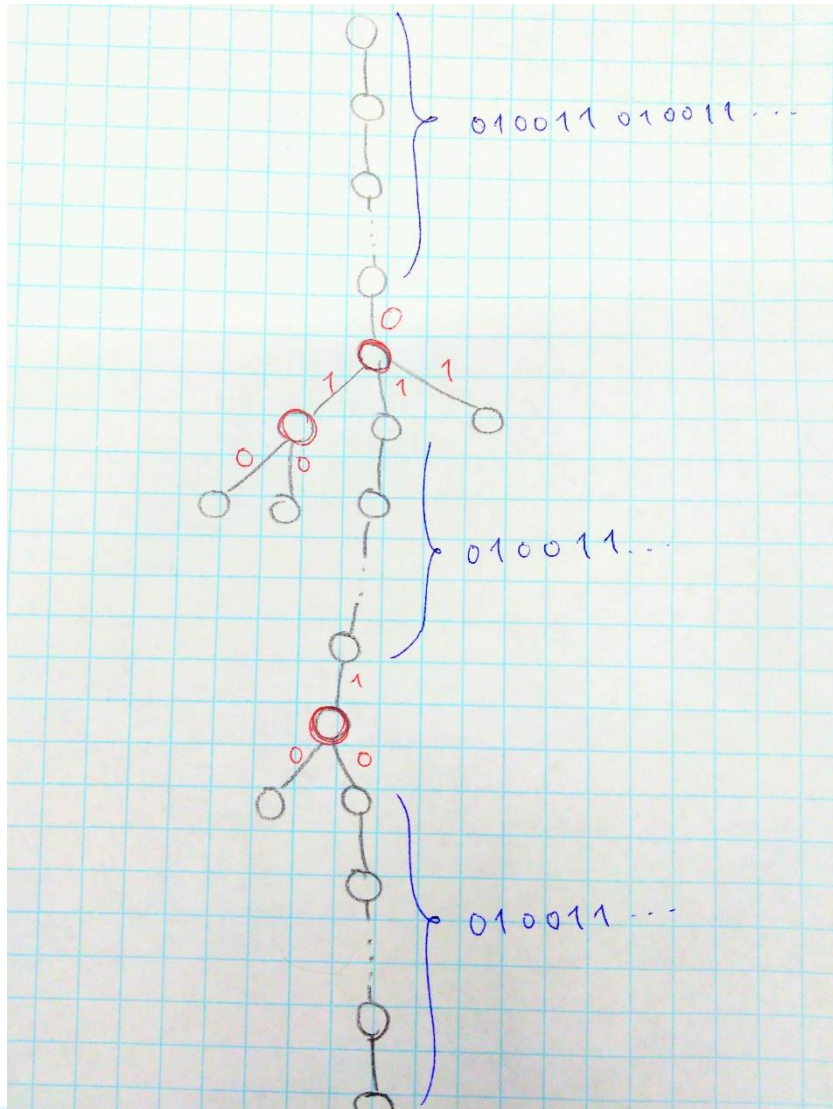
# パスのとき



- 逆走しても  $B/2$  歩以内に気づけば OK



- 読んでいる方向がわかる 0, 1 の列を書く  
◻ 可能な最小周期は 6



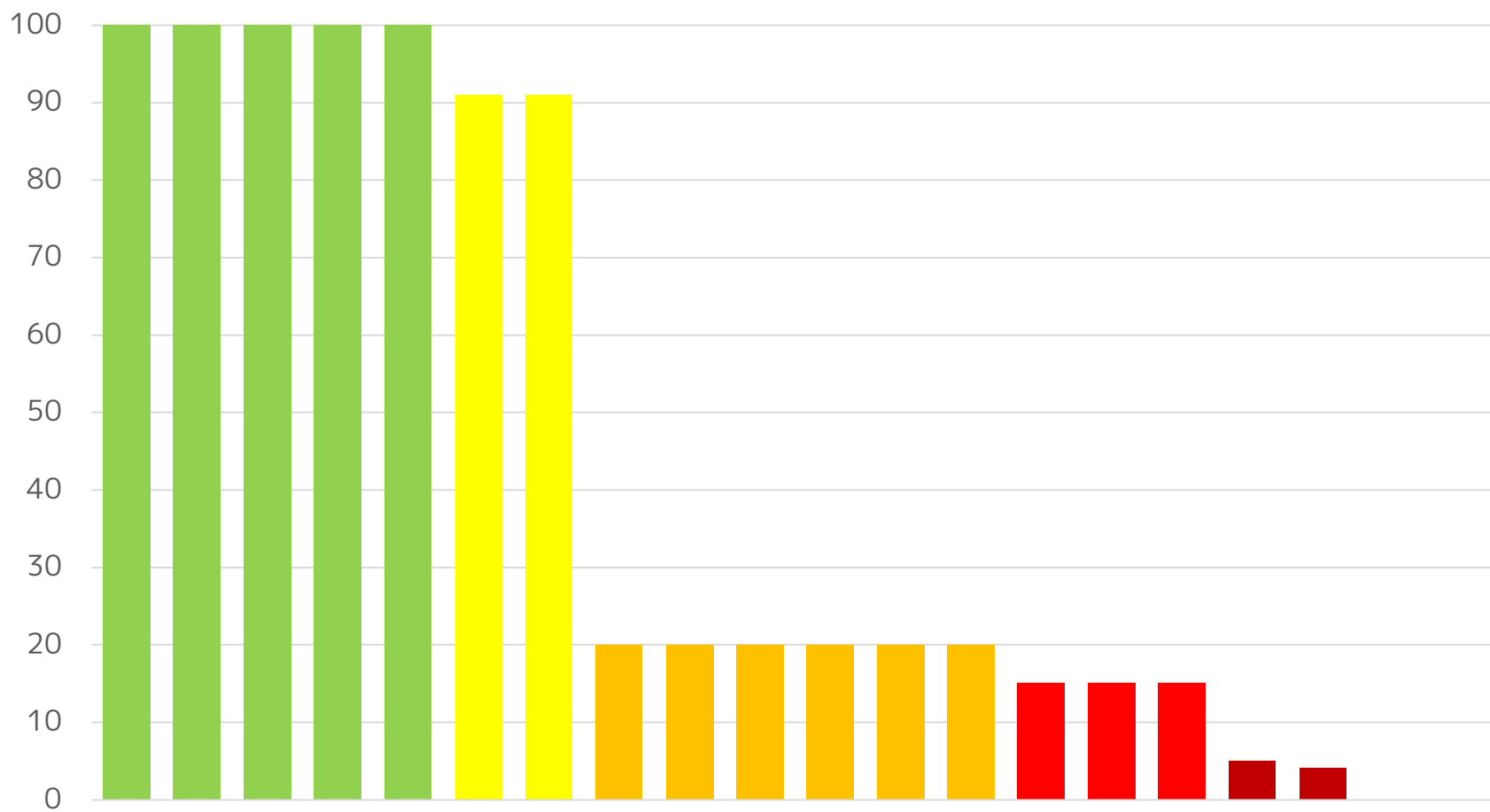
- 次数 2 以外まわりは根方向だけ違う値に
- 残るパスに 010011
- 次数 2 以外の頂点に着いたら、以降根方向がわかる
- 次数 2 にしばらくいたら逆走に気づく 



# 小課題 {5, 6, 7} (85 点)

- 次数  $\geq 3$  の頂点から次数 2 の部分へ入るところをいい感じの書き方で繋げる
  - 根からの距離にこだわらない
- 5 文字読めれば方向がわかる
- 情報をすべて使うと 3 歩で 5 文字読める
  - 0 歩の時点で 2 文字読めている

# 得点分布



# Capital City

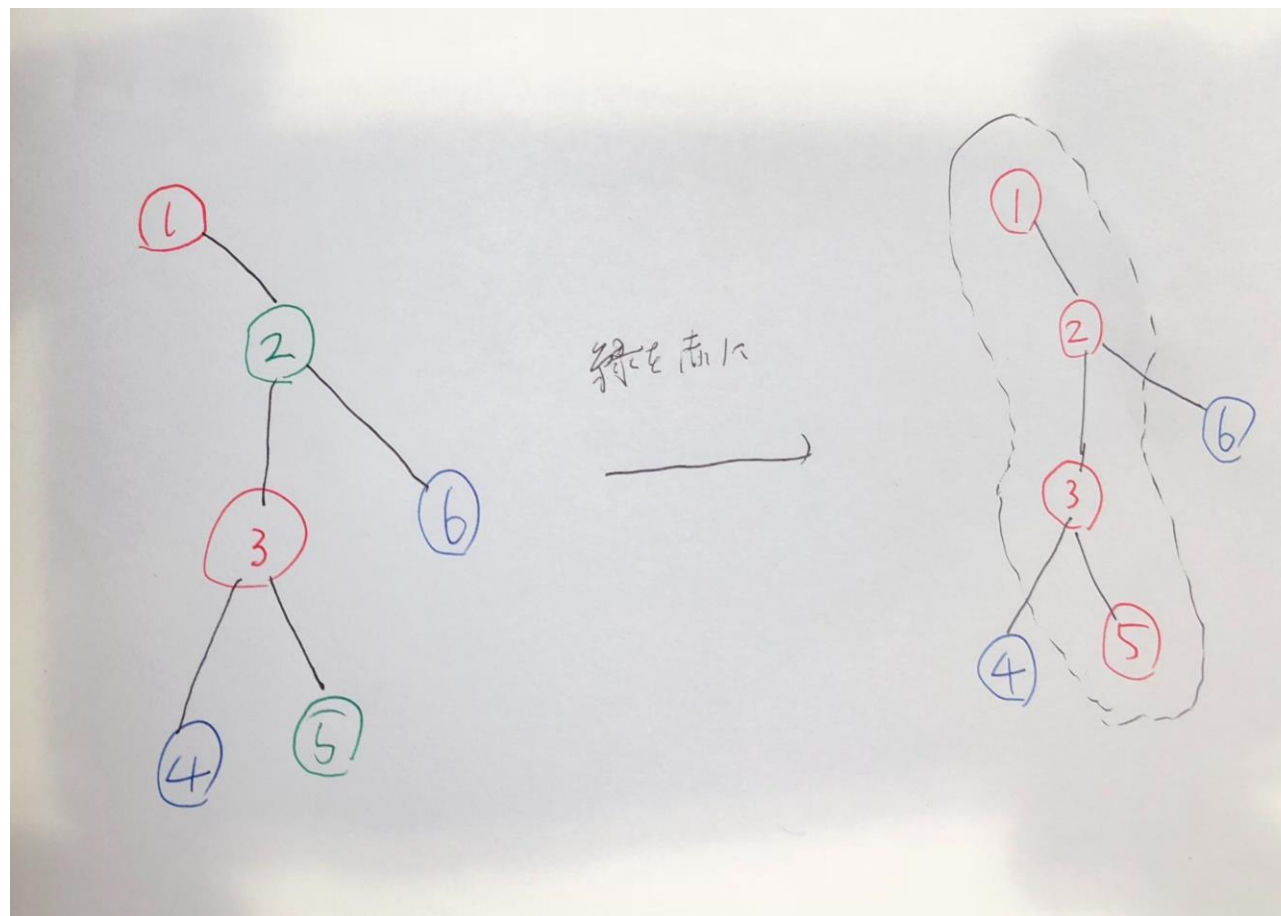
伊佐 碩恭 (HIR180)



# 問題概要

- 木が与えられ，各頂点は何らかの都市に属している
- 都市と都市の合併を繰り返すことにより，「その都市に含まれるどの2頂点間も，その都市に含まれる頂点のみを通過することで行き来できるような都市」を作りたい
- 最小で何回合併を行えば良いですか？

# 入力例1



# 言い換え

- グラフ理論の通称に従って，町を頂点と言い換えます
- この問題において，任意の連結な部分木が首都として指定できるので，首都を連結な部分木と同義のものとして解説します

# 小課題1

- $N \leq 20$
- 「最終的に合併されて1つの都市になる」ような都市の集合を全て探索することができる
- 集合が決まれば、いずれかの都市に含まれる頂点たちが全域木を成すかどうかで判定できる
- $O(2^N * N)$
- 1点

## 小課題2

- $N \leq 2000$
- 最終的に首都に含まれる頂点を1つ決め打つと，首都に必ず入らなくてはならない都市の集合が定まる（決め打った頂点を $v$ とすると， $v$ と同じ色の頂点と， $v$ を結ぶパス上にある頂点の都市もまた最終的に首都に含まれる… という操作を新たに都市が追加されなくなるまで繰り返す）
- 重要な性質として，この操作によりできた首都は頂点 $v$ を含む首都の中で**極小**であることが容易にわかる

## 小課題2

- この性質より，各頂点を始点として前ページの処理を行い最小値を取れば答えが求まる
- $O(N^2)$
- 10点

# 小課題3

- 木は直線グラフ
- 色々な方針があります
  1. 数列の問題だと思う
  2. 分割統治
  3. Segment tree状に辺を貼る

# 小課題3 – 方針1

- 左端Lを決め打ちすると、右端Rに対する条件は以下のようになる:
  - $[L, R]$ に含まれる要素について、「自分と同じ都市の頂点で、左方向に一番近い頂点番号」が存在しないか、すべてL以上
  - $[L, R]$ に含まれる要素について、「自分と同じ都市の頂点で、右方向に一番近い頂点番号」が存在しないか、すべてR以内
- これは、segtreeやstd::setを使うことで実装できる
- 30点



## 小課題3 – 方針2

- 今、頂点 $[l,r]$ に対して考えているとする
- 頂点  $m = (l+r)/2$  を含む場合と含まない場合に分ける
- 含む場合は、小課題2の方針で $O(r-l)$ で解ける
- 含まない場合は、 $[l,m-1]$   $[m+1,r]$ に分割して同じ処理をする
- 困るのは今見ている範囲外の町を含めないといけない場合
  - 実は、そのような場合は処理をやめて良い (今までに求めた首都のいずれかを包含する  $\rightarrow$  その時に求めた首都を包含する)

## 小課題3 – 方針3

- 都市 $c$ を首都に含めることにすると，都市 $c$ に含まれる町のうち一番左にあるものを $L_c$ ，一番右にあるものを $R_c$ とすると， $[L_c, R_c]$ の町は全て首都に含める必要が生じる
- これを素直なグラフの問題に落とすと，都市 $c$ に対応する頂点から町 $L_c, \dots, R_c$ に対応する頂点へ辺を張り，できたグラフを強連結成分分解して，出次数0の強連結成分に対して含まれる都市に対応する頂点の個数の最小値 - 1を求める問題になる

## 小課題3 – 方針3

- このままでは辺数が $O(N^2)$ なので正答できない
- が、区間に辺を張る という特性を使いオーダーが落とせる
- Segment tree (葉ノードが1つの町に対応する頂点) に対し、親から子へ有向辺を張ったグラフを考えると、 $[l, r]$ に辺を張る という処理は $O(\log N)$ 個の区間に対して辺を張れば良くなる
- よって、強連結成分分解は頂点数と辺数の和に線形な時間で求まるため、全体で $O(N \log N)$ 時間の解法になる
- 類題: [https://atcoder.jp/contests/arc069/tasks/arc069\\_d](https://atcoder.jp/contests/arc069/tasks/arc069_d)

# 満点解法

- こちらも色々な方針があります
  1. 重心分解 (<- 小課題3の方針2の拡張)
  2. 強連結成分分解 (<- 小課題3の方針3の拡張)

# 満点解法 – 方針1

- 小課題2の性質を思い出す
  - 「最終的に首都に含まれる頂点を1つ決め打つと，首都に必ず入らなくてはならない都市の集合が定まる」
- 頂点を決め打つ時に性質の良い頂点を選ぶと上手くいく??
- **重心分解**

# 満点解法 - 方針1

- 木の重心を使うと決め打てば、今の部分木サイズに線形な計算量で最小値が求まる
- 木の重心を使わないケースは、重心分解して分かれた連結成分ごとに考えれば良い
- 今見ている部分木の外に頂点が存在する都市を使わないと行けなくなった場合は、計算を中止して良い（今までの重心のいずれかは使わないといけなくなる -> その場合の首都を包含するため、このケースは考えなくて良い）
- $O(N \log N)$ 時間で解ける

# 満点解法 – 方針2

- 小課題3の時は区間に辺を張る だったので簡単だったが、今回は木のパス上に辺を張る必要がある
- 木を直線グラフに分解する方法があったような…?
  - Heavy Light Decomposition
- $O(\log N)$ 個の直線グラフに対し、区間に辺を張る 問題に変換できた
- 先ほどと同様にして、 $O(N \log^2 N)$ 時間で解ける
  - 実装によってはTLEするかも

# 満点解法? - 乱択アルゴリズム

- 小課題2と同様にして、最終的に首都に含まれる頂点 $v$ を決め打って極小な首都を求めることを考える
- 新たに都市 $c$ を追加しないといけなくなった場合、**都市 $c$ に含まれる頂点を使うと決め打って計算したことがある**ならば処理を打ち切って良い（その場合の首都を包含するから）
- 使うと決め打つ頂点の順番や、首都を求める際に次に追加すべき都市を見つける際の処理をランダムに行うことによって、非常に高速に動作した
- 解析は簡単ではない
  - 少なくとも期待値で  $O(N \log N)$  時間かかるケースは存在する

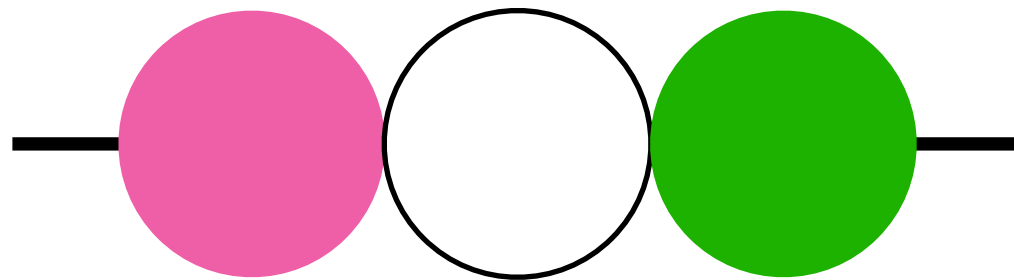


# LGM

Legendary danGo Maker

# 問題概要

- できる限り団子をたくさん作ってください
- 8方向



# 方針

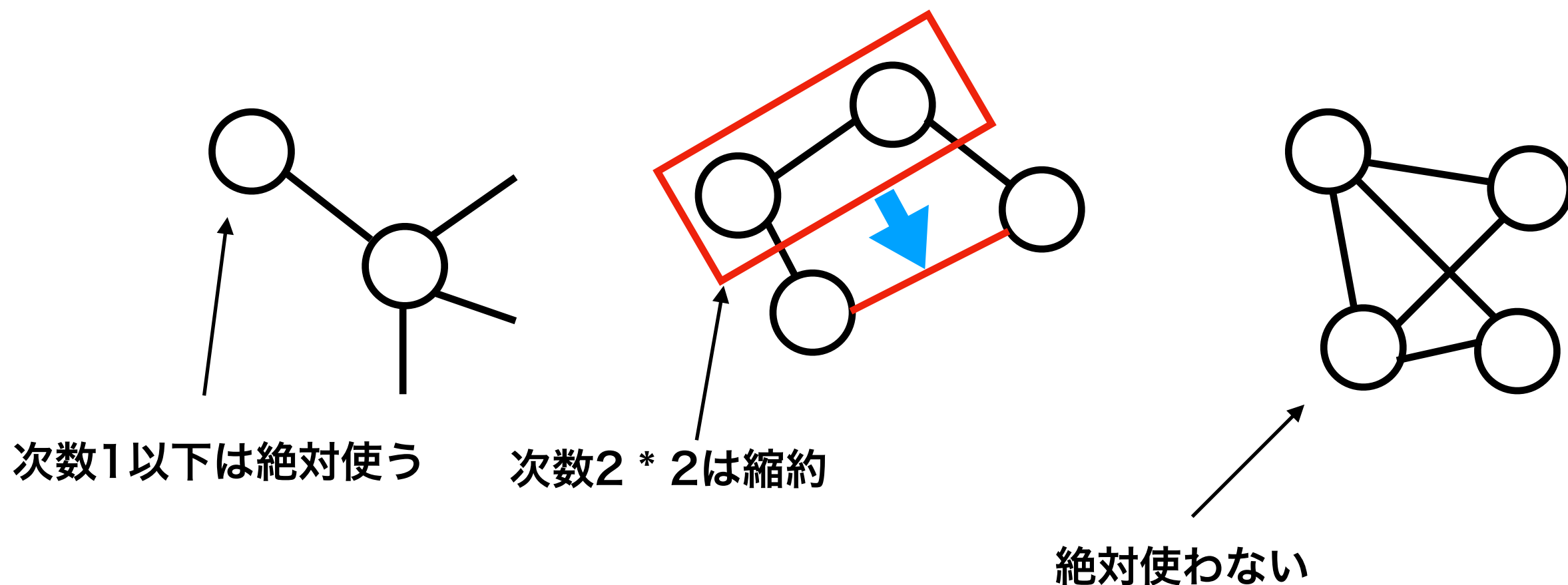
- 解説もクソもないので、方針をいろいろ紹介します
- 焼きなまし
- 最大独立点集合
- IPソルバー(gurobi)
- Akiba Takuya, Yoichi Iwata. Branch-and-reduce exponential/FPT algorithms in practice: A case study of vertex cover

# 焼きなまし

- 団子の串追加
- 団子の串削除
- で焼きなましをします 頑張って高速化したりしよう！

# 端から取ってく

- 団子の串を頂点としてみると、この問題は最大独立点集合問題として見ることができます
- 端っから結構削れます



# IPソルバー

- 実はIPという形で定式化できます

IPソルバーで出来ること(の一部)

- bool変数を用意する(串を使うかどうか)
- $x_i + x_j \leq 1$  (串を同時に使えない)

- IPソルバーはたくさんあります(が、無料と有料にすごい性能差が…)
- GLPK(無料)
- CPLEX(有料, 大学生なら学部によっては無料)
- Gurobi(有料, 大学生なら無料) → **全ケース最適解!**

# Akiba Takuya, Yoichi Iwata. Branch-and-reduce exponential/FPT algorithms in practice: A case study of vertex cover

- という論文があります
- Vertex Cover (= 最大独立点集合) を解いています
- [https://github.com/wata-orz/vertex\\_cover](https://github.com/wata-orz/vertex_cover) に実装があります
- 05以外は最適解が出てきます

# Treatment Project

解説

nxteru



# 概要

- ・ コストを使って、ある日にある連続した区間を治療する計画が与えられる
- ・ 治療された区間は日経つにつれて縮んでいく
- ・ 全体を治療するのにかかるコストの最小値を求めよ

## 小課題1 (4点)

- $T_i = 1$
- コスト付きの区間が与えられるので、全体を覆うコストの最小値を求める問題

## 小課題1 (4点)

- 計画をRでソート
- $dp[i]$ ...i番目までの計画を使いR[i]まで覆うようにするときのコストの最小値とする
- $dp[i] = \min\{dp[j] \mid j < i \text{ かつ } L[i] - 1 \leq R[j]\} + C[i]$
- このjは連続した範囲になっているので最小値を求めるsegment treeで高速化できる
- $O(M \log M)$

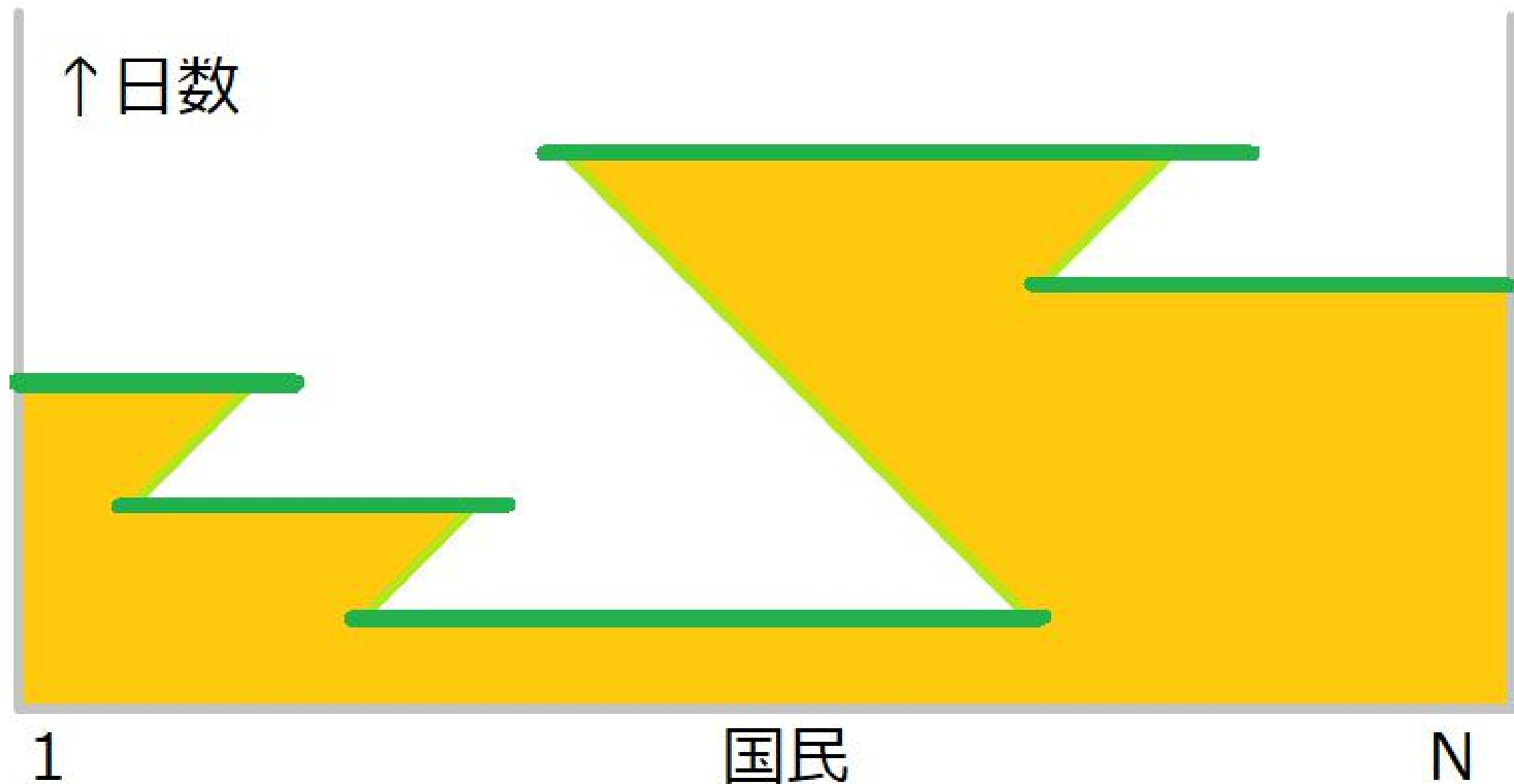
## 小課題2 (5点)

- $M \leq 16$
- $2^M$ 通りの計画の採用の仕方をすべて調べることができる
- 採用した計画で条件を満たしているかの判定が少し煩雑
- $M$ が小さいので少しさぼっても大丈夫
- 自分の実装だと  $O(2^M M^2)$

## 小課題3 (30点)

- $M \leq 5000$
- ここからちゃんと考えないといけない
- 小課題2でめんどくさかった条件を満たすかの判定について考える
- 条件を満たす計画の採用の仕方はどのような計画の集合だろう？

# 条件を満たす計画の集合の例



# 考察

- ・ 治療された区間が狭まりながら計画どうしがつながって感染した領域と治療された領域を分ける境界を形成している
- ・ つまり国民1を含む計画からつながっている計画をたどって国民Nを含む計画まで行ければよい

→最短経路問題になった

### 小課題3 (30点)

- ・ 計画aから計画bに行けるときaからbにコスト  $C[b]$  の有向辺をつくる
- ・ その条件は  $L[b] \leq R[a] - |T[b] - T[a]| + 1$  と表せる
- ・ これをすべての計画の組について調べて辺をはり、国民1を含む計画から国民Nを含む計画までの最短距離を求める
- ・ ダイクストラ法などで  $O(M^2)$

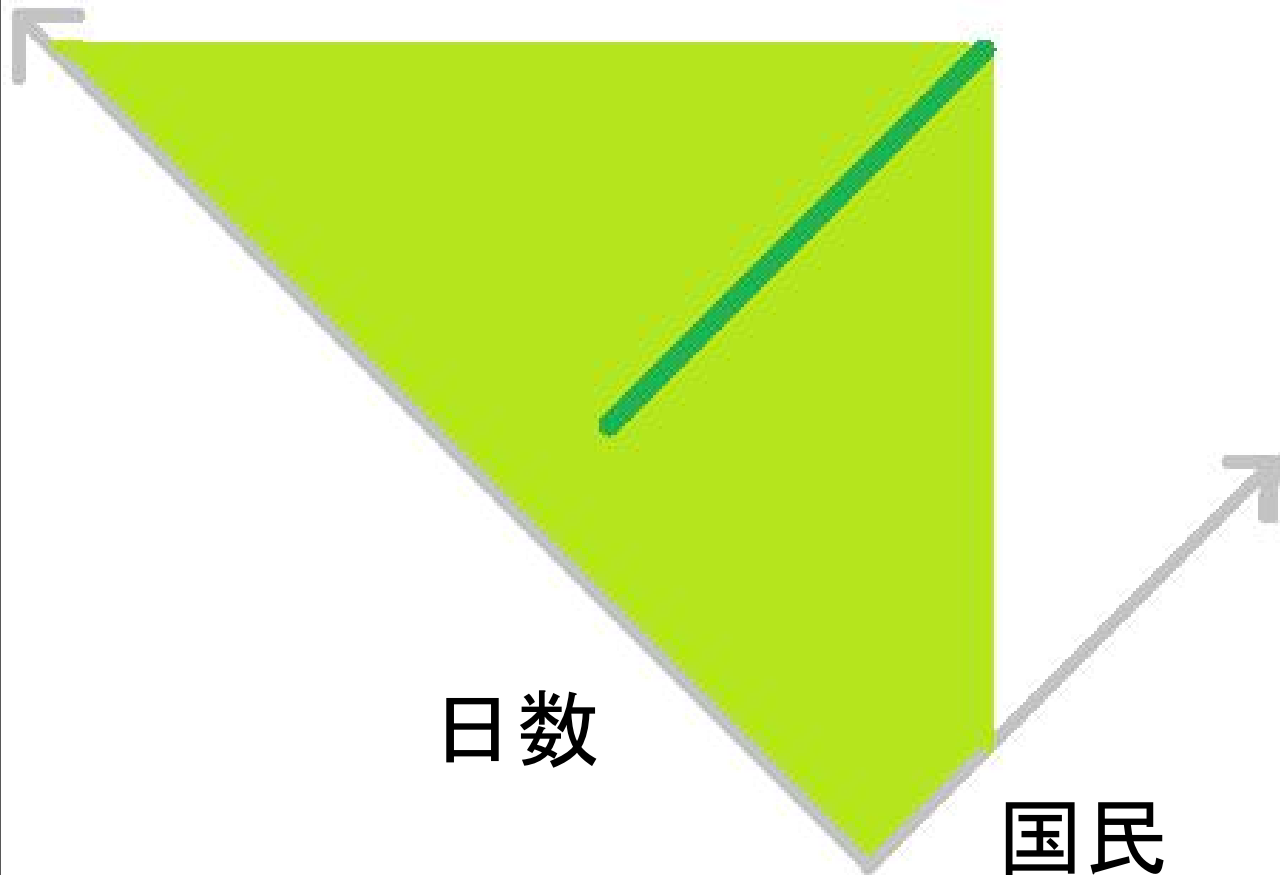


## 小課題4 (61点)

- ・ 追加の制約なし ( $M \leq 100000$ )
- ・ 辺をすべてはることもできない
- ・ 小課題3では最短経路問題をそのまま解いただけ
- ・ 今まで使っていなかったこの問題特有の性質を考える

## いい性質①

- 45度回転させるとある計画(下図の緑)から辺がでる計画は下図のような長方形領域に左端がある計画になる



$$X[i] = L[i] - T[i], Y[i] = L[i] + T[i]$$

とすると

aからbに遷移できるのは

$$X[b] \leq R[a] - T[a] + 1$$

かつ

$$Y[b] \leq R[a] + T[a] + 1$$

のとき

## いい性質②

- ・ ある計画aに入ってくる辺のコストはすべて $C[a]$ で辺の始点に関わらず同じ
  - ・ 問題を解くときにダイクストラ法を使う
  - ・ するとある計画aの距離が最初に更新された段階でそれが最短距離になっている
- 一度距離を更新したらその計画はもう更新しなくていい！

## 小課題4 (61点)

- ・ この2つの性質を使ってpriority\_queueを使ったダイクストラ法をする
- ・ 距離の更新をするときに長方形領域の点を見て更新しそれ以降見ないようにする
- ・ 必要なのは長方形領域の点を列挙して削除するデータ構造

## 小課題4 (61点)

- ・ 計画を $X$ でソート、その状態から $Y$ の値でマージソートする過程をseg木に保存
- ・ seg木の葉ノードは1つの計画を持っていて葉から順に2つの子ノードを $Y$ の値でマージしていく
- ・ 各ノードは $X$ の対応する範囲の計画を、 $Y$ でソートした列を持っている
- ・  $O(M \log M)$ で構築できる

## 小課題4 (61点)

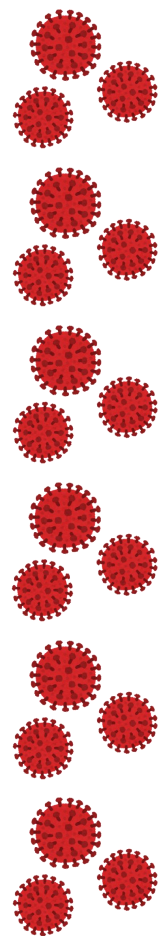
- ・ダイクストラ法を行い、ある計画 $a$ が取り出されたらseg木の $X \leq R[a] - T[a] + 1$ に対応するノードにアクセスする
  - ・アクセスしたノードでは $Y$ の小さい順に見て $Y \leq R[a] + T[a] + 1$ を満たす間、その計画を取り出していく(取り出したらseg木から削除する)
- 更新するべき計画の列挙、削除ができた

## 小課題4 (61点)

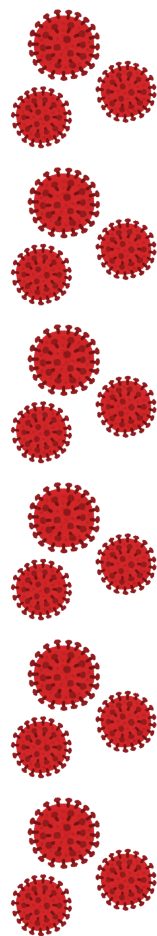
- priqueueに同じ計画が2度入らないようにすると
- priqueueの計算量は $O(M \log M)$
- seg木のノードへのアクセスは合計 $O(M \log M)$
- seg木内の計画は $O(M \log M)$ 個あるが同じ計画が2回以上見られることはないので合計 $O(M \log M)$

→満点!!

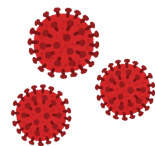
# 得点分布



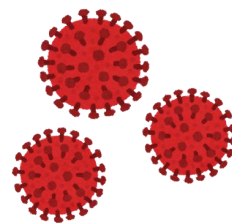
0点



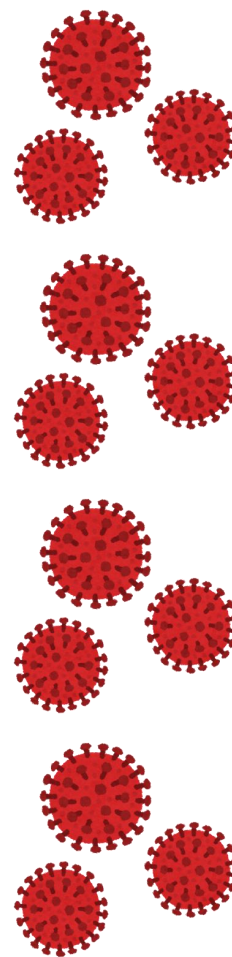
4点



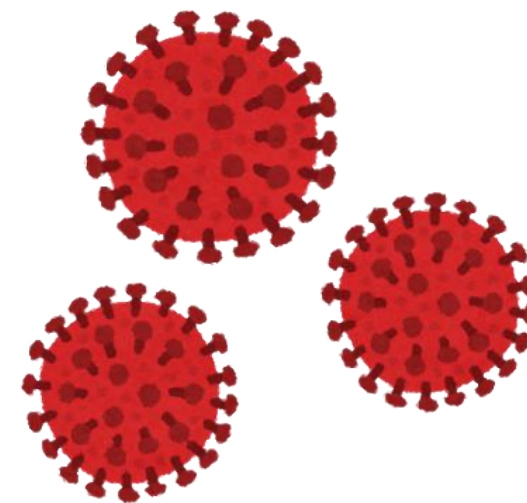
9点



35点



39点



100点